Two-stage scheduling and process-mode selection for dual-source mixed-flow PECVD rotating-chamber cluster tools

Lizhen Shao[a,*], Lincong Wei[a], Wei Wang[a]

[a]*Department of Control Science and Engineering, University of Science and Technology Beijing, No. 30 Xueyuan Road, Haidian District, Beijing 100083, China*

Abstract

PECVD cluster tools with multi-position rotating chambers require coordinated scheduling of product wafers, dummy wafers, and periodic cleaning. Material origins, processing modes, and robot-hand eligibility jointly affect finite-batch operation. This study develops a two-stage mixed-integer linear programming model that jointly determines wafer pairing, dummy allocation, chamber assignment, resource order, and event times. The model represents single- and dual-source supply routes and distinguishes clean and dirty hands of the vacuum transfer robot. The first stage minimizes product-return makespan; the second refines waiting and cadence while retaining the makespan bound and principal discrete structure. Structured initialization and learning-enhanced cut selection support time-limited solving. All clean/dirty-hand experiments yield independently validated physical schedules. In a four-product structural comparison, reassigning dummy identities and reoptimizing timing creates 111 s of concurrent chamber processing and reduces makespan from the original fixed-structure optimum of 532 s to 403 s, a reduction of 24.25%. Across six single- and dual-source configurations under the same 300-s total solve budget, timing refinement reduces total transport-route waiting by 8.40%–29.21% and shortens actual product makespan. Mode sweeps for lots of 8, 16, and 24 wafers further compare feasible schedules across cleaning intervals. These results quantify the effects of dummy-loan arrangements and timing optimization within a structure on finite-batch operation.

Keywords: semiconductor manufacturing system, cluster tool, dual-source material flow, operational decision, mixed-integer linear programming, learning-enhanced exact optimization

1. Introduction

Plasma-enhanced chemical vapor deposition (PECVD) is used in semiconductor thin-film fabrication. Cluster tools (CTs) connect process modules, robots, and load locks into a continuous wafer-handling route. Processing and transport times, load-lock switching, cleaning, and residency constraints jointly determine feasible operation and equipment output [1]. Three-dimensional device structures and multilayer integration increase demands on deposition uniformity and processing capability [2]. Multi-space process modules and multi-finger robots provide a structural basis for parallel processing within a tool [3, 4].

This study considers a PECVD CT with four-position rotating chambers. Rotation changes the positions accessible to the robot, allowing processing to alternate with paired loading and unloading. Products enter from atmospheric load ports, pass through alignment and load-lock transitions, and reach the chambers through the vacuum transfer robot. Dummies are stored in vacuum and reused to satisfy the full-occupancy and cleaning requirements of the prescribed recipe. We call the separate atmospheric product supply and vacuum dummy supply dual-source operation, and their interleaved use of shared robots and chambers mixed flow.

Full occupancy couples product supply with dummy reuse. Dummies fill positions when products cannot occupy the entire chamber. Cleaning occupies dummies and chambers and changes their subsequent availability. Product arrival therefore does not alone permit processing: dummy availability, robot service order, and cleaning boundaries must also be satisfied. The two material routes in Fig. 1 require coordinated scheduling.

Two preset modes are considered: 4×1 parallel processing and 2×2 chained processing. The former processes four wafers synchronously once; the latter uses paired transfers and rotation to give each product two consecutive exposures. Each product recipe is prescribed in the instance and remains unchanged during scheduling; mode comparisons use separate instances with equal cumulative nominal processing time. Pair composition, chamber assignment, and cycle placement determine dummy requirements, chamber loads, and cleaning triggers, while shared robots constrain when these arrangements can be executed. This study therefore addresses joint pairing, chamber assignment, and event timing for given recipes and source configurations, and compares operating modes across lot sizes and cleaning intervals.

CT studies established analytical relationships between processing, handling, and throughput [5] and used linear programming, heuristics, and timed Petri nets to obtain schedules under residency constraints [6–8]. Later models address revisit-

*Corresponding author.

Email addresses: lshao@ustb.edu.cn (Lizhen Shao), m202420836@xs.ustb.edu.cn (Lincong Wei)

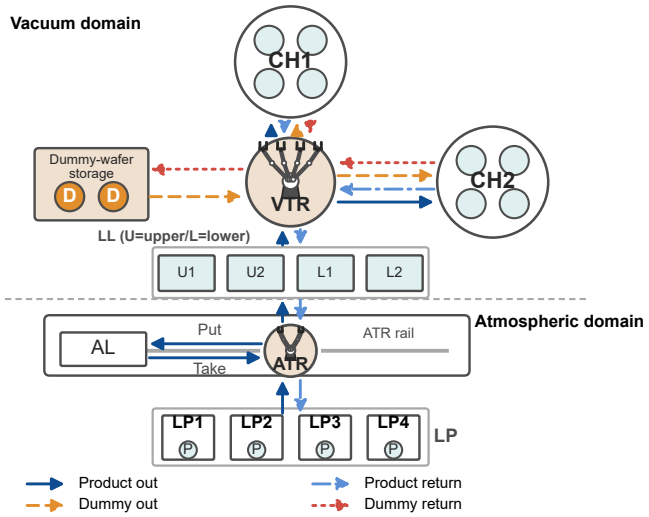


Figure 1: Dual-source mixed-flow rotating-chamber CT topology and wafer routes.

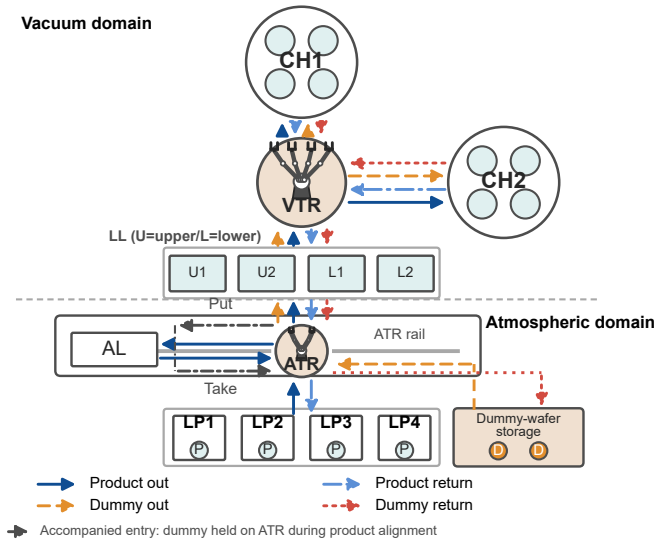


Figure 2: Single-source equipment topology and material routes. Dummy storage is beside LP4. Products undergo AL alignment. A separately supplied dummy moves directly from ATR to the upper load lock; when collected with a product, it remains on ATR during the AL visit and enters the lock with the aligned product. Both types reach the chambers through VTR and return through the lower load lock and ATR.

ing processes, multiple wafer types, and selectable modules [9–11]. Clean/dirty-hand separation and steady-state analysis of four-finger robots and four-space modules, together with joint optimization of two-type wafer releases and handling, supports more complex configurations [4, 12]. Finite batches additionally involve chain initialization, termination, and partial-batch filling, making dummy return times direct constraints on subsequent processing.

Periodic-cleaning scheduling uses task-sequence enumeration, partial loading and timing control, virtual-wafer occupancy, and chamber-state-dependent triggers to coordinate maintenance and production [13–16]. Shared-buffer synchronization and dynamic action screening address deadlock during failure shutdown and noncyclic operation [17, 18]; waiting constraints also affect equipment output and upstream control [19]. Here, cleaning simultaneously changes chain segmentation, dummy occupancy, and vacuum-robot order. Scheduling product processing or cleaning independently would omit these dependencies.

These material and resource constraints define a mixed-integer scheduling problem. Learning can select CT actions directly or assist mathematical optimization. Reinforcement learning with look-ahead search supports noncyclic CT control [20]; reinforcement-learning selection, cut ranking, and look-ahead imitation select candidate inequalities in MILP [21–23]. HEM jointly determines cut counts and ordered sequences [24]. We retain this hierarchical decomposition and use equipment-variable structure for cut selection while representing process feasibility in the MILP.

This study jointly models product supply, dummy reuse, complete processing chains, and cleaning boundaries, and separately evaluates schedule construction and search strategies through paired experiments. Its contributions are:

1. A joint pairing, dummy-allocation, chamber-assignment, and timing model coordinates cross-LP pairing of same-recipe products, product–dummy pairing, both supply routes, and clean/dirty VTR-hand eligibility. Cycle occupancy, first/last exposures, and side assignments directly represent full occupancy, complete chains, and cleaning boundaries; a controlled dummy-identity comparison tests the structural benefit of cross-chamber loan arrangements.

2. A makespan-first two-stage MILP refines waiting and cadence while retaining the first-stage makespan bound and principal discrete structure, with schedules evaluated across lot sizes and cleaning intervals.

3. SPBS initialization and role-aware hierarchical cut selection are embedded in the solver. Equal-budget comparisons separately evaluate initialization, timing refinement, and cut selection in terms of feasible-schedule availability and solution quality.

Section 2 defines material routes, process constraints, and the two-stage model. Section 3 presents initialization and cut selection. Section 4 analyzes mode choice, schedule quality, and computational performance. Section 5 concludes.

2. Problem description and two-stage scheduling model

2.1. Equipment configuration and scheduling task

Given product counts and recipes at LP1–LP4, dummy inventory, and the initial equipment state, scheduling jointly determines wafer pairing, dummy identities, chamber assignment, shared-resource order, and the start and finish times of loading, alignment, handling, rotation, processing, cleaning, and return actions. Products depart from LPs and return after processing. Dummies depart from the storage region specified by

the source configuration, perform filling or cleaning tasks, and return. Single-source storage is beside LP4 in the atmospheric domain; dual-source storage is in the vacuum domain. Robot service, chamber occupancy, and dummy inventory couple the routes.

Inputs comprise action durations, resource capacities, residency limits, cleaning intervals, and dummy counts. Stage one minimizes the final product return time. Stage two improves waiting and cadence under the resulting makespan bound. Each feasible schedule is represented by an action sequence specifying wafers, resources, and start and completion times for performance evaluation and checks of precedence and resource conflicts.

The cleaning interval satisfies $H \geq 2$, and the shared inventory contains at least four dummies. Action durations are deterministic within a solve. No buffers are assumed beyond LPs, aligners, load locks, rotating chambers, and vacuum storage, so each wait corresponds to a material location and resource-occupancy interval.

2.2. PW-pair modeling for the two process modes

To represent wafer-level residency and paired handling together, shared transfer and chamber occupancy are modeled at the pair level, while alignment and route timing retain wafer-level events.

A wafer pair has two member positions and may contain two products, a product and a dummy, or two dummies. Product load ports ℓ_w and recipes m_w are inputs; products with the same recipe may be paired across LPs. Members of candidate pairs $\mathcal{P}$ are selected through binary variables a_{wpr} for product membership at position $r \in \{0, 1\}$, d_{pr} for dummy occupancy, and v_p for activation:

$$\begin{aligned} \sum_{p,r} a_{wpr} &= 1 \quad (w \in \mathcal{W}), \\ \sum_{w} a_{wpr} + d_{pr} &= v_p \quad (p \in \mathcal{P},\ r = 0, 1). \end{aligned} \tag{1}$$

Let $\mathcal{M} = \{\zeta, \psi, \mathrm{cl}\}$, where ζ denotes 4×1, ψ denotes 2×2, and cl denotes cleaning. The set $C(m)$ contains chambers compatible with mode m. Binary variables μ_{pm} and z_{pc} select the pair recipe and chamber:

$$\begin{aligned} \sum_{m} \mu_{pm} &= v_p, \qquad \sum_{c} z_{pc} = v_p, \\ a_{wpr} &\leq \mu_{p,m_w}, \\ z_{pc} + \mu_{pm} &\leq 1 \qquad\qquad (c \notin C(m)). \end{aligned} \tag{2}$$

Equations (1) and (2) assign each product once and fill both positions of every active pair. For $c \notin C(m)$, the final inequality prohibits selecting mode m and chamber c simultaneously. Product recipe m_w and source configuration σ are inputs; pairing, chamber assignment, and processing order are decisions. Binary quantities B_p and B_p^{ψ} identify product-containing pairs and pairs containing 2×2 products:

$$\begin{aligned} B_p \leq \sum_{w,r} a_{wpr} \leq 2B_p, &\qquad B_p + \mu_{p,\mathrm{cl}} \leq 1, \\ B_p^{\psi} \leq B_p, \quad B_p^{\psi} \leq \mu_{p\psi}, &\quad B_p^{\psi} \geq B_p + \mu_{p\psi} - 1. \end{aligned} \tag{3}$$

Dummy-only pairs provide filling or cleaning; product-containing pairs cannot perform cleaning. Membership, cycle occupancy, and event links directly represent the joint structure without enumerating complete schedules. For 4×1 and 2×2 product counts N_ζ, N_ψ, respectively, candidate capacities are

$$\begin{aligned} T &= N_\zeta + 2N_\psi, \qquad J_{\max} = \lfloor T/(H-1) \rfloor, \\ P_{\max} &= N + T + 2J_{\max}, \qquad Q_{\max} = T + J_{\max}. \end{aligned} \tag{4}$$

T bounds productive cycles because every productive cycle contains a product, and each 4×1 or 2×2 product appears once or twice, respectively. Each cleaning follows at least $H-1$ productive cycles, giving $J_{\max}$. There are at most N product-containing pairs, one dummy-only support pair per productive cycle, and two dummy-only pairs per cleaning, giving $P_{\max}$. Each chamber receives $Q_{\max}$ candidate cycles, with a shared bound on total productive and cleaning cycles. These capacities bound candidate indices; optimization determines how many are activated. Prefix activation of pair labels and canonical member ordering remove interchangeable labels without prescribing physical processing order.

A product follows LP–ATR–AL–ATR–upper load lock–VTR–rotary chamber–VTR–lower load lock–ATR–LP: the ATR first places the wafer on the aligner, then retrieves it for transfer into the vacuum domain. Under dual-source supply, a dummy travels from vacuum storage through the VTR to the chamber and returns through the VTR after use; under single-source supply, it moves from storage beside LP4 through ATR directly to the upper load lock. When collected with a product, the dummy accompanies the ATR to AL and remains on its gripper until the aligned product is retrieved; both then enter the upper load lock together. The dummy itself requires no alignment. A regular all-product 4×1 batch contains two product PW pairs, hence four product wafers, and processes all four positions synchronously once before last-in–first-out unloading; dummy or mixed PW pairs fill any remaining positions. In 2×2, every product-containing PW pair occupies two adjacent process cycles so that each of its product members is processed twice. For three product PW pairs, a dummy PW pair D is loaded first, the chamber rotates, and P1 is loaded before D+P1 undergoes chamber processing. Chamber processing then alternates with "chamber rotation + chain slide", changing the occupancy successively to P1+P2, P2+P3, and P3+D; P1, P2, and P3 are each processed twice before all wafers are unloaded. Figures 3 and 4 show the event sequences.

Table 1 lists sets and inputs, and Table 2 lists decision and auxiliary variables. Linking constraints define auxiliary variables for positions, attributes, and timing relationships. Membership, chamber assignment, and cycle occupancy jointly determine the processing structure. Operation times use s_o, e_o; cycle and transfer times alias the corresponding physical events.

2.3. Material-source configuration and time relations

Let $\mathcal{S} = \{A, V\}$ denote atmospheric and vacuum supply regions, respectively. The input parameter $\sigma \in \{0, 1\}$ specifies single-source and dual-source configurations, respectively. LP1–LP4 are individual atmospheric cassette locations; each

Table 1: Core sets and input parameters

Symbol	Type	Definition and role
$\mathcal{P}, \mathcal{M}$	Sets	Candidate PW pairs and modes: ζ for 4×1, ψ for 2×2, and cl for cleaning.
$\ell_w, m_w, C(m)$	Inputs/sets	Product LP, prescribed mode $m_w \in \{\zeta, \psi\}$, and chambers compatible with mode m.
$\mathcal{W}, \mathcal{I}$	Sets	Product wafers and material units, indexed by w, i; $N = \lvert\mathcal{W}\rvert$.
$\mathcal{S}, \sigma, \chi_{is}$	Sets/binary inputs	Supply regions $\mathcal{S} = \{A, V\}$; $\sigma = 0/1$ denotes single/dual source; χ_{is} identifies the source.
$\mathcal{P}^{\zeta}, \mathcal{P}^{\psi}$	Sets	Product-containing PW pairs for 4×1 and 2×2, indexed by p.
$\mathcal{C}, \mathcal{L}$	Sets	Chambers {CH1, CH2} (index c) and two sides $\mathcal{L} = \{1, 2\}$ (index l).
$\mathcal{B}_c, \mathcal{K}_c, \mathcal{Q}_c$	Index sets	Batches, chain positions, and unified candidate cycles, indexed by b, k, q.
$\mathcal{E}_c, \mathcal{V}_c$	Sets	Cleaning segments and processing tasks, indexed by e, v.
$\mathcal{O}, \mathcal{A}$	Sets	Physical operations and process-precedence arcs; operations are indexed by o.
$\mathcal{R}$	Set	Shared resources, indexed by r.
$\mathcal{J}_c^D, \mathcal{T}_c$	Sets	Dummy demands and reusable physical dummy wafers, indexed by j, τ.
H, d_o, r_c	Integer/time inputs	Maximum cycles before cleaning, operation duration and chamber half-rotation time.
$\delta_{oo'}, \rho^r_{o\underline{o}'}, \rho^{\mathrm{VTR}}_{cc'}$	Time inputs	Minimum process lag and resource repositioning or reset time.
$M, \overline{R}_{wr}$	Time bounds	Valid big-M and maximum wafer residency in a restricted region.
Γ_o	Set input	Eligible units for occupancy task o; dirty VTR hands D_1, D_2 and clean hands C_1, C_2.
$\lambda_1, \ldots, \lambda_4$	Nonnegative weights	Weights for total wait, maximum wait, cadence deviation and soft-limit excess.
$\bar{\omega}_a, \varepsilon_C$	Nonnegative time inputs	Soft waiting limit and stage-two makespan tolerance.
$\mathcal{A}^{\mathrm{int}}, \mathcal{A}_{is}(\gamma)$	Conditional arc sets	Common chamber precedences and routes for material i, source s, and transfer choices γ.
$\mathcal{F}$	Feasible region	Joint region defined by pairing, cycles, complete routes, and resource constraints.
$\mathcal{D}_{\mathrm{fix}}, \mathcal{Q}^w, \mathcal{I}_h$	Index sets	Stage-two fixed variables, waiting terms and cadence differences.
v; FL, BL, P, BU, FU	Stage indices	Front load, back load, processing, back unload and front unload.
pick, ALput, ALpick, LLput	Event labels	Source pickup, placement at AL, retrieval from AL, and placement in the upper load lock; stage subscripts of s_{wa}, e_{wa}.
$\mathrm{in}(w, r), \mathrm{out}(w, r)$	Event maps	Entry and exit events of wafer w for restricted region r.
N_ζ, N_ψ; $T, J_{\max}$	Integer inputs/bounds	4×1 and 2×2 product counts, and bounds on productive cycles and cleanings.
$P_{\max}, Q_{\max}, \overline{T}, d_m$	Capacity/time inputs	Candidate pair/cycle capacities, time horizon, and processing duration of mode m.

product wafer has a prescribed load port. Supply regions and cassette indices therefore describe different levels. Products satisfy $\chi_{iA} = 1$ and $\chi_{iV} = 0$, whereas dummies satisfy $\chi_{iA} = 1 - \sigma$ and $\chi_{iV} = \sigma$. The source-incidence parameters satisfy

$$\sum_{s\in\mathcal{S}} \chi_{is} = 1, \qquad i \in \mathcal{I}. \tag{5}$$

Equation (5) specifies the supply region of each material unit. In the single-source configuration, products occupy LP1–LP4 and dummies occupy atmospheric storage beside LP4. Both use ATR and the load-lock interface, but only products require alignment. In the dual-source configuration, products originate from LP1–LP4 while dummies originate from vacuum storage. Single-source thus denotes atmospheric supply of both wafer types, rather than a single cassette.

Let $\mathcal{A}^{\mathrm{int}}$ denote common in-chamber precedences and $\mathcal{A}_{is}(\gamma)$ the supply/return arcs of material unit i from source s under transfer choices γ. Material type and paired handling jointly determine the activated routes:

$$\mathcal{A} = \mathcal{A}^{\mathrm{int}} \cup \bigcup_{i\in\mathcal{I}} \bigcup_{\substack{s\in\mathcal{S}:\\ \chi_{is}=1}} \mathcal{A}_{is}(\gamma). \tag{6}$$

Equation (6) selects routes by material unit. Products pass through LP, ATR, AL, ATR, the upper load lock, VTR, and a chamber. A separately supplied single-source dummy follows atmospheric storage, ATR, the upper load lock, VTR, and a chamber. When collected with a product, the dummy remains on an ATR gripper during the AL visit; this creates gripper occupancy but no dummy placement, retrieval, or alignment event at AL. Joint placement in the load lock follows product retrieval from AL. Both atmospheric wafer types return through VTR, the lower load lock, and ATR to their original storage locations. Dual-source dummies travel only between vacuum storage and a chamber through VTR.

For each candidate operation o, activation and duration satisfy

$$0 \le s_o \le M b_o, \qquad 0 \le e_o \le M b_o, \qquad e_o = s_o + d_o b_o. \tag{7}$$

Equation (7) sets inactive operation times to zero and gives an active operation duration d_o. It applies to robot moves, alignment, pump/vent, rotation, product processing, dummy transport, and cleaning.

For every required process arc $(o, o') \in \mathcal{A}$,

$$b_{o'} \le b_o, \qquad s_{o'} \ge e_o + \delta_{oo'} - M(2 - b_o - b_{o'}). \tag{8}$$

Equation (8) activates a successor only with its predecessor and, when both exist, starts it after the predecessor and minimum lag. These arcs connect product routes, dummy routes, and load–process–unload chains.

2.4. Joint cycle occupancy and rotating-chamber transfers

Let $\mathcal{Q}_c = \{1, \ldots, Q_{\max}\}$ index candidate cycles of chamber c, including both production modes and cleaning. Binary variables U_{cq}, K_{cqm}, and X_{pcq} select cycle activation, cycle type, and participating PW pairs. Every active cycle contains two pairs,

Table 2: Core decision variables, auxiliary variables, and time aliases

Symbol	Type	Definition and role
a_{wpr}, d_{pr}, v_p	Binary variables	Product membership, dummy positions and pair activation; member positions $r = 0, 1$.
μ_{pm}, z_{pc}	Binary variables	Pair recipe and chamber choices; product recipes are inputs.
$x^{\zeta}_{pcbl}, x^{\psi}_{pck}$	Binary auxiliary variables	Batch/chain positions determined by cycle occupancy and sides; additionally fixed in the layout benchmark.
$g_{j\tau}$	Binary variables	Binary assignment of demand j to physical dummy wafer τ.
$y^{r}_{oo'}$	Binary variables	Binary order: operation o precedes o' on shared resource r.
$h_{o\ell}, \gamma_{pa}$	Binary variables	Physical slot or hand ℓ occupancy and paired transfer on route a; w_1, w_2 are pair members and $a =$ in denotes joint entry.
b_o	0–1 auxiliaries	0–1 operation activation, constrained by the layout and transfer choices.
s_o, e_o	Nonnegative continuous	Nonnegative operation start and end times, with $e_o = s_o + d_o b_o$.
$C_w, C_{\max}$	Nonnegative quantities	Product-return time $C_w = e_{w,\text{return}}$ and lot makespan.
$\omega_\alpha, L_{\text{sum}}, L_{\max}$	Nonnegative continuous (s)	Waiting duration for item α, total waiting, and an upper bound on maximum individual waiting.
Δ_{ceq}	Real auxiliary variable (s)	Difference between consecutive start intervals centered on process q in chamber c, cleaning segment e; may have either sign.
v_{ceq}	Nonnegative auxiliary (s)	Upper bound on $\lvert\Delta_{ceq}\rvert$, representing cadence variation linearly.
η_α	Nonnegative auxiliary (s)	Upper bound on the positive excess of wait α above its soft limit $\bar{\omega}_\alpha$.
$\xi, \widehat{\xi}, \widehat{C}$	Vector/stage constants	All variables, stage-one feasible solution and product makespan; the latter two are refinement references.
$z_{ce}, a^{\zeta}_{cbe}, a^{\psi}_{cqe}$	Binary auxiliary variables	Cleaning-segment activation and batch/exposure membership.
$a_{vce}, N_{ce}, q^{\text{cl}}_{ce}$	0–1 quantities/count	Task-to-segment membership, segment cycle count and cleaning activation.
$s^{\text{cl}}_{ce}, e^{\text{cl}}_{ce}, s^{\text{cl},v}_{ce}, e^{\text{cl},v}_{ce}$	Time-variable aliases	Start/end times of a complete cleaning task and its stages.
s_v, e_v, s_{wa}, e_{wa}	Time-variable aliases	Times of processing task v and wafer w on route segment a.
$x, s^{\text{ref}}, e^{\text{ref}}$	Selector/time aliases	Generic stage selector and reference start/end times of the selected stage.
h_{ceq}	Nonnegative time alias (s)	Processing start of cycle q in chamber c, cleaning segment e; reuses the corresponding s_o.
Φ	Objective expression	Weighted sum of total wait, maximum wait, cadence variation, and optional soft-limit excess.
B_p, B^{ψ}_p	Binary auxiliary variables	Pair p contains products or 2×2 products.
U_{cq}, K_{cqm}, X_{pcq}	Binary variables	Cycle activation, cycle type, and pair participation.
A_{pcq}, Z_{pcq}	Binary auxiliary variables	First and last participation of pair p in a cycle.
$X^{P}_{pcq}, A^{\psi}_{pcq}, Z^{\psi}_{pcq}$	Binary auxiliary variables	Product-pair occupancy and first/last exposures of 2×2 product pairs.
S_{cq}, E_{cq}, n_{cq}	Times/integer count	Processing start/end and productive-cycle count since cleaning.
$f_{pcq}, \kappa_{cq}, \upsilon_{cq}$	Binary quantities	Front-side pair, retention from the preceding cycle, and full departure.
$s^{\text{load}}_p, e^{\text{load}}_p$, $s^{\text{unload}}_p, e^{\text{unload}}_p$	Time aliases	Start/end of pair loading and unloading by VTR.
$s^{\text{rot}\pm}_{cq}, e^{\text{rot}\pm}_{cq}$	Time aliases	Start/end of pre- and post-cycle half-rotations.
$\gamma^{\text{early}}_p, \gamma^{\text{late}}_p$	Binary variables	Direct dummy entry before product alignment, or pickup after product placement at AL.

with compatible recipes and chambers:

$$\begin{aligned} &\sum_m K_{cqm} = U_{cq}, && \sum_p X_{pcq} = 2U_{cq}, \\ &X_{pcq} \le z_{pc}, && X_{pcq} + \mu_{pm} \le 1 + K_{cqm}, \\ &\sum_p X^P_{pcq} \ge K_{cq\zeta} + K_{cq\psi}, && U_{c,q+1} \le U_{cq}, \\ &\sum_{c,q} U_{cq} \le Q_{\max}, && \sum_{c,q} K_{cq,\text{cl}} \le J_{\max}. \end{aligned} \tag{9}$$

The indicator X^P_{pcq} identifies a participating product-containing pair through

$$X^P_{pcq} \le X_{pcq}, \quad X^P_{pcq} \le B_p, \quad X^P_{pcq} \ge X_{pcq} + B_p - 1. \tag{10}$$

Every productive cycle therefore contains a product pair, whereas cleaning contains only dummy pairs. Prefix activation eliminates empty indices without prescribing the order of different recipes.

Binary variables A_{pcq}, Z_{pcq} identify the first and last cycles of pair p in chamber c. With boundary values

$$X_{pc0} = X_{pc,Q_{\max}+1} = 0,$$

$$\begin{aligned} &A_{pcq} \ge X_{pcq} - X_{pc,q-1}, && A_{pcq} \le X_{pcq}, \\ &A_{pcq} \le 1 - X_{pc,q-1}, \\ &Z_{pcq} \ge X_{pcq} - X_{pc,q+1}, && Z_{pcq} \le X_{pcq}, \\ &Z_{pcq} \le 1 - X_{pc,q+1}, \\ &\sum_{c,q} X_{pcq} = v_p + B^{\psi}_p, && \sum_{c,q} A_{pcq} = \sum_{c,q} Z_{pcq} = v_p, \\ &\sum_p A^{\psi}_{pcq} \le 1, && \sum_p Z^{\psi}_{pcq} \le 1. \end{aligned} \tag{11}$$

Here $A^{\psi}_{pcq} = A_{pcq} \wedge B^{\psi}_p$ and $Z^{\psi}_{pcq} = Z_{pcq} \wedge B^{\psi}_p$ use the three-inequality linearization in Eq. (10). Each active pair enters and exits once, so occupancy is contiguous. Its total occupancy $v_p + B^{\psi}_p$ gives one cycle for a 4×1 product pair and two adjacent exposures in the same chamber for a 2×2 product pair. At most one 2×2 product pair starts or finishes exposure in each cycle, producing the alternating exchange chain. Each dummy-only support or cleaning pair occupies one cycle.

Cycle processing starts and ends at S_{cq}, E_{cq}; d_m is the pro-

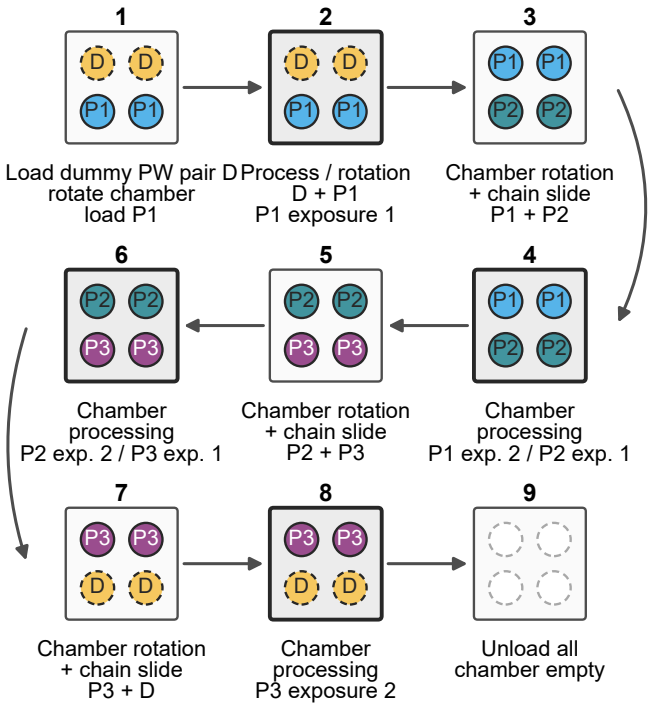


Figure 3: 2×2 chained processing. A PW pair may be P+P, D+D, or P+D; here each Pi is a P+P product PW pair and D is a D+D dummy PW pair. Step 1 loads D, rotates the chamber, and loads P1; Step 2 performs chamber processing on D+P1; Step 3 applies chamber rotation + chain slide to obtain P1+P2. Repeating "chamber processing–chamber rotation + chain slide" makes P1, P2, and P3 participate in two adjacent process cycles, so every product wafer is processed twice before all wafers are unloaded.

cessing duration of mode m and $\overline{T}$ is a valid time horizon:

$$\begin{aligned} E_{cq} - S_{cq} &= \sum_m d_m K_{cqm}, \\ 0 \le S_{cq}, E_{cq} &\le \overline{T} U_{cq}, \\ S_{cq} &\ge E_{c,q-1} - M(1 - U_{cq}) \quad (q > 1). \end{aligned} \tag{12}$$

Processing durations are fixed, with waiting confined to intervals between operations. Defaults are 180 s for 4×1, 90 s per 2×2 exposure, and 500 s for cleaning treatment; loading, unloading, and rotation retain their separate event durations.

Let binary f_{pcq} identify the pair on the robot-accessible front side during processing, κ_{cq} indicate retention of one pair from the preceding cycle, and υ_{cq} indicate complete departure after

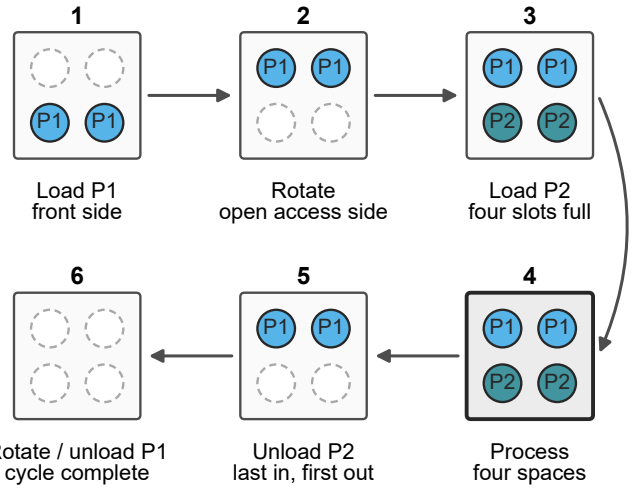


Figure 4: 4×1 parallel processing. A regular all-product batch uses two product PW pairs, comprising four product wafers, and receives one synchronous full-occupancy process before last-in–first-out unloading; a dummy or mixed PW pair can complete the final short batch.

the current cycle:

$$\begin{aligned} \kappa_{cq} &= \sum_p (X_{pcq} - A_{pcq}), \\ \upsilon_{cq} &= U_{cq} - \sum_p (X_{pcq} - Z_{pcq}), \\ f_{pcq} &\le A_{pcq}, \qquad \sum_p f_{pcq} = U_{cq}, \\ f_{pc,q-1} &\ge X_{pcq} - A_{pcq} \quad (q > 1), \\ f_{pcq} &\ge B_p + K_{cq\psi} + X_{pcq} - 2 - \kappa_{cq}. \end{aligned} \tag{13}$$

The newly loaded pair occupies the front; a retained pair rotates to the back and must have occupied the front in the preceding cycle. At a fresh 2×2 chain head, the product pair loads last at the front. Pair loading/unloading times reuse their physical events. Pre- and post-cycle half-rotations, denoted rot− and rot+, are activated by U_{cq} and υ_{cq}, respectively, and last r_c. Loading is linked by

$$\begin{aligned} X_{pcq} = 1 &\Rightarrow S_{cq} \ge e_p^{\text{load}}, \quad s_p^{\text{unload}} \ge E_{cq}, \\ A_{pcq} = 1,\ f_{pcq} = 0 &\Rightarrow e_p^{\text{load}} \le s_{cq}^{\text{rot}-}, \\ A_{pcq} = 1,\ f_{pcq} = 1 &\Rightarrow s_p^{\text{load}} \ge e_{cq}^{\text{rot}-}, \quad S_{cq} = e_p^{\text{load}}, \\ \kappa_{cq} = 1 &\Rightarrow s_{cq}^{\text{rot}-} \ge E_{c,q-1}. \end{aligned} \tag{14}$$

When all pairs depart, post-cycle rotation starts after processing. Unloading after a pair's final cycle satisfies

$$\begin{aligned} Z_{pcq} = 1,\ f_{pcq} = 1 &\Rightarrow e_p^{\text{unload}} \le s_{cq}^{\text{rot}+}, \\ Z_{pcq} = 1,\ f_{pcq} = 0,\ \upsilon_{cq} = 1 &\Rightarrow s_p^{\text{unload}} \ge e_{cq}^{\text{rot}+}, \\ Z_{pcq} = 1,\ f_{pcq} = 0,\ \upsilon_{cq} = 0 &\Rightarrow s_p^{\text{unload}} \ge e_{c,q+1}^{\text{rot}-}. \end{aligned} \tag{15}$$

If the next cycle retains one pair, incoming loading waits for removal of the replaced back-side pair. If no pair is retained, loading waits for both pairs from the preceding cycle to be

unloaded. Indicator constraints or valid big-M inequalities linearize these conditions; VTR exclusion additionally constrains the exchange. Waiting is allowed and remains separate from fixed rotation and handling durations.

In a selected schedule, $x^{\zeta}_{pcbl}, x^{\psi}_{pck}$ record 4×1 batch sides and 2×2 product-pair chain positions derived from X, A, Z, f and cycle types. Batches, chains, and cleaning segments describe the schedule and the fixed-layout benchmark without introducing independent pairing decisions. Fixed-layout experiments additionally fix positions and service orders and restrict dummy filling to the last batch: a 4×1 batch preceding another batch of that type contains two product pairs. The joint model permits dummy filling in any admissible productive cycle.

2.5. Cleaning and reusable dummy wafers

Integer n_{cq} counts productive cycles since the last cleaning at the end of cycle q. With terminal candidate values $U_{c,Q_{\max}+1} = K_{c,Q_{\max}+1,\psi} = 0$,

$$\begin{aligned}
&0 \le n_{cq} \le HU_{cq}, \qquad n_{c0} = 0,\\
&K_{cq,\mathrm{cl}} = 1 \Rightarrow n_{cq} = 0,\\
U_{cq} = 1,\ &K_{cq,\mathrm{cl}} = 0 \Rightarrow n_{cq} = n_{c,q-1} + 1, \quad n_{c,q-1} \le H-1,\\
&K_{cq,\mathrm{cl}} = 1 \Rightarrow n_{c,q-1} \ge H-1, \quad n_{c,q-1} + K_{c,q+1,\psi} \ge H,\\
&n_{cq} \le H - 1 + HU_{c,q+1},\\
&K_{c1,\mathrm{cl}} = 0, \qquad K_{cq,\mathrm{cl}} + K_{c,q-1,\mathrm{cl}} \le 1 \quad (q>1).
\end{aligned} \tag{16}$$

Production increments the count and cleaning resets it. Production cannot continue at count H; early cleaning at $H-1$ is allowed only when the next cycle is 2×2. The last active cycle has count at most $H-1$, requiring terminal cleaning when the final production segment reaches H. Cleaning cannot interrupt a pair's two exposures because occupancy and cycle type jointly satisfy Eqs. (9) and (11).

Cleaning segments e are induced by cleaning boundaries in the cycle-type sequence. Indicators z_{ce}, $a^{\zeta}_{cbe}, a^{\psi}_{cqe}$, and q^{cl}_{ce} identify active production segments, batch/exposure membership, and subsequent cleaning; N_{ce} counts productive cycles. Internal segments end with cleaning, as does a final segment reaching H. These quantities describe event windows and vary with cycle decisions.

Let $\mathcal{V}_c$ be the union of 4×1 batches and 2×2 cycles and let a_{vce} take the corresponding a^{ζ} or a^{ψ}. Cleaning order and the physical cleaning chain satisfy

$$\begin{aligned}
s_v &\ge e^{\mathrm{cl}}_{c,e-1} - M(1-a_{vce}) && (e>1),\\
e_v &\le s^{\mathrm{cl}}_{ce} + M(1-a_{vce}) + M(1-q^{\mathrm{cl}}_{ce}),\\
s^{\mathrm{cl,BL}}_{ce} &\ge e^{\mathrm{cl,FL}}_{ce} + r_c - M(1-q^{\mathrm{cl}}_{ce}),\\
s^{\mathrm{cl,P}}_{ce} &\ge e^{\mathrm{cl,BL}}_{ce} - M(1-q^{\mathrm{cl}}_{ce}),\\
s^{\mathrm{cl,BU}}_{ce} &\ge e^{\mathrm{cl,P}}_{ce} - M(1-q^{\mathrm{cl}}_{ce}),\\
s^{\mathrm{cl,FU}}_{ce} &\ge e^{\mathrm{cl,BU}}_{ce} + r_c - M(1-q^{\mathrm{cl}}_{ce}).
\end{aligned} \tag{17}$$

Equation (17) starts a segment after the preceding cleaning and finishes it before the next cleaning. A cleaning task loads both chamber sides with dummies, rotates, processes, and unloads in reverse, thereby occupying the VTR, chamber, and four physical dummies rather than adding virtual downtime.

Each dummy member position defines a loan demand $j = (p, r)$, activated directly by d_{pr}. This covers mixed product–dummy pairs, dummy-only filling pairs and cleaning pairs. The selected configuration specifies supply and return events; a physical-dummy loan extends from storage pickup through completed return.

Chambers access a shared physical inventory: $\mathcal{T}_c$ denotes the identities available to chamber c, and loans of the same identity are mutually exclusive across chambers. Each active demand selects one physical dummy from inventory:

$$\sum_{\tau\in\mathcal{T}_c} g_{j\tau} = d_{pr}, \qquad j=(p,r)\in\mathcal{J}^D_c. \tag{18}$$

Equation (18) assigns every active demand exactly once. The identity selection $g_{j\tau}$ also activates its full-loan resource-occupancy interval. Mutual exclusion orders repeated loans of the same physical dummy, so it can be reused only after unloading and return to storage. Identity selection determines which cross-chamber loans require ordering. Even with identical product pairing, chamber assignment, and cycle structure, different dummy identities can change the intervals of concurrent chamber processing. Section 4.8 quantifies this effect while controlling product structure.

2.6. Robots, load locks, chambers, and time resources

Shared-resource orders comprise prescribed and selectable precedences. Operations on process-precedence arcs and prescribed service chains follow their specified order; other conflicting operations are coordinated by ordering variables. The fixed-layout pure 4×1 experiments prescribe aligner and load-lock service orders. When the candidate batch count does not exceed the cleaning threshold, VTR loading and unloading also follow a prescribed batch-service chain. These prescribed orders apply to the fixed-layout experiments; in the joint model, ordering variables select the remaining resource precedences.

ATR has two hands, VTR has four hands, and each rotating chamber has four wafer positions, giving physical capacities of two, four, and four wafers, respectively. Each robot executes one coordinated handling action at a time, and each chamber executes one loading, unloading, rotation, or processing stage at a time. ATR, VTR, and each chamber actuator are therefore unary action resources; wafer-level hand assignments and pair occupancy represent their physical carrying capacities. AL is also a unary action resource. The upper load lock has slots U1 and U2, and the lower load lock has slots L1 and L2; each dummy identity is a reusable resource. Resource assignment is defined for individual occupancy intervals: a wafer-position occupancy, a dummy loan, or an exclusive actuator use selects one eligible unit. Denoting this resource-occupancy task by o,

$$\sum_{\ell\in\Gamma_o} h_{o\ell} = b_o. \tag{19}$$

Equation (19) assigns each active resource-occupancy task exactly one eligible unit. The binary indicator b_o activates the task,

$h_{o\ell}$ selects unit ℓ, and Γ_o contains eligible units; all assignments are zero for an inactive task. Wafers transferred as a pair retain individual wafer-position occupancies, with synchronized actions governed by Eq. (23). PW-pair occupancy enforces the four-position chamber requirement; an actuator's exclusion unit does not represent wafer capacity. Equation (20) prevents overlapping occupancy of a shared unit. For a singleton Γ_o, the identity $h_{o\ell} = b_o$ can be substituted directly without a separate assignment variable. Dummy-identity occupancy reuses g_{jr} rather than introducing duplicate identity decisions.

Upper-LL occupancy extends from ATR placement to completion of VTR pickup plus reset; lower-LL occupancy extends from VTR placement to completion of ATR pickup plus reset. Reset restricts the next use of that slot and may overlap onward robot motion after pickup.

The VTR has two dirty hands, D_1, D_2, and two clean hands, C_1, C_2. For an unprocessed product transferred from the upper load lock to a chamber, $\Gamma_o = \{D_1, D_2\}$; for a fully processed product transferred from a chamber to the lower load lock, $\Gamma_o = \{C_1, C_2\}$. Dummy wafers may use $\Gamma_o = \{D_1, D_2, C_1, C_2\}$ on either transfer leg. Each occupancy interval extends from pickup start through placement completion and retains the same physical hand selected by Eq. (19). Equation (20) excludes overlapping use of that hand. The separation between hand-occupancy intervals is zero; robot travel remains governed by the actuator-level travel constraints. Dirty and clean hands therefore each carry at most two wafers, with dummies competing for available hands of either type. A 2×2 product remains inside the chamber between its consecutive exposures and is retrieved by a clean hand only after the second exposure. During a paired exchange, dirty hands may hold the incoming product pair while clean hands retrieve the completed pair; every dummy must likewise occupy a specific available hand.

If operations o and o' may compete for slot ℓ of resource r, then

$$\begin{aligned} s_{o'} &\geq e_o + \rho^r_{oo'} - M(1 - y^r_{oo'}) - M(1 - h_{o\ell}) - M(1 - h_{o'\ell}), \\ s_o &\geq e_{o'} + \rho^r_{o'o} - M y^r_{oo'} - M(1 - h_{o\ell}) - M(1 - h_{o'\ell}). \end{aligned} \tag{20}$$

Equation (20) makes $y^r_{oo'}$ choose one order whenever two actions use the same slot and includes robot repositioning or equipment reset through $\rho^r_{oo'}$. It applies to ATR, AL, both load locks, VTR, rotary chambers, and individual dummies, preventing concurrent robot actions and reuse before release.

Travel times are inputs for each location pair. When z_{pc} determines an operation's chamber, the corresponding travel lag is activated for each candidate chamber combination. For active conflicting chamber operations o, o' serving pairs p, p', for example,

$$z_{pc} = z_{p'c'} = 1,\ y^{\mathrm{VTR}}_{oo'} = 1 \ \Rightarrow\ s_{o'} \geq e_o + \rho^{\mathrm{VTR}}_{cc'}. \tag{21}$$

Here $\rho^{\mathrm{VTR}}_{cc'}$ is the prescribed inter-chamber travel time; the reverse order is analogous. Fixed-location operations use Eq. (20) directly.

Process precedences and selected resource orders form a directed event graph. A cycle of conflicting operations with positive durations or transitions would require a start time to be strictly later than itself and is excluded by timing feasibility. Together with explicit slot occupancy, full chamber occupancy, and complete dummy return, these constraints exclude circular waiting among the modeled operations.

An individual product or dummy stage must coincide with its selected batch, chain position, or cleaning window. For selection x and reference stage $[s^{\mathrm{ref}}, e^{\mathrm{ref}}]$,

$$\begin{aligned} -M(1 - x) \leq s_o - s^{\mathrm{ref}} \leq M(1 - x), \\ -M(1 - x) \leq e_o - e^{\mathrm{ref}} \leq M(1 - x). \end{aligned} \tag{22}$$

Equation (22) synchronizes the individual action with its physical stage when $x = 1$ and releases the candidate relation when $x = 0$, connecting pair positions, product routes, dummy demands, and chamber windows.

If pair p uses one paired transfer on a compatible atomic handling leg a at a common location, the two wafer stages are synchronized:

$$\begin{aligned} -M(1 - \gamma_{pa}) \leq s_{w_1 a} - s_{w_2 a} \leq M(1 - \gamma_{pa}), \\ -M(1 - \gamma_{pa}) \leq e_{w_1 a} - e_{w_2 a} \leq M(1 - \gamma_{pa}). \end{aligned} \tag{23}$$

Equation (23) means that $\gamma_{pa} = 1$ moves a complete pair in one non-preemptive robot action. If $\gamma_{pa} = 0$, the wafers still occupy the same robot separately and are ordered by Eq. (20). Actuator exclusion counts coordinated actions: the two wafer events of a paired transfer represent one action, so Eq. (20) is not activated between them. The shared action remains mutually exclusive with other actions. Individual holding intervals are retained and physical-hand exclusion always applies; synchronizing actions does not merge wafer capacity.

For a single-source product–dummy pair, let w_1 denote the product and w_2 the dummy. The choice $\gamma_{p,\mathrm{in}}$ activates accompanied entry after their collection. This route-linking choice does not synchronize pickups at different storage locations. Source pickup, AL placement, AL retrieval, and upper-load-lock placement are labeled pick, ALput, ALpick, LLput, respectively. Let $\gamma^{\mathrm{early}}_p, \gamma^{\mathrm{late}}_p$ select direct dummy entry before the AL visit or pickup after product placement at AL. Together with accompanied entry, exactly one is selected:

$$\begin{aligned} \gamma^{\mathrm{early}}_p + \gamma^{\mathrm{late}}_p + \gamma_{p,\mathrm{in}} &= 1, \\ \gamma^{\mathrm{early}}_p = 1 &\Rightarrow e_{w_2,\mathrm{LLput}} \leq s_{w_1,\mathrm{ALput}}, \\ \gamma^{\mathrm{late}}_p = 1 &\Rightarrow s_{w_2,\mathrm{pick}} \geq e_{w_1,\mathrm{ALput}}, \\ \gamma_{p,\mathrm{in}} = 1 \ \Longrightarrow &\begin{cases} e_{w_2,\mathrm{pick}} \leq s_{w_1,\mathrm{ALput}}, \\ s_{w_2,\mathrm{LLput}} = s_{w_1,\mathrm{LLput}}, \\ e_{w_2,\mathrm{LLput}} = e_{w_1,\mathrm{LLput}}, \\ s_{w_1,\mathrm{LLput}} \geq e_{w_1,\mathrm{ALpick}}. \end{cases} \end{aligned} \tag{24}$$

The dummy is already carried when the product is placed at AL and enters the upper load lock simultaneously with the aligned product. It occupies one ATR gripper continuously from source pickup through load-lock placement. The product releases its gripper while on AL and occupies it again upon retrieval. Independent dummy entry instead directly links source pickup

to load-lock placement. Neither alternative activates dummy alignment at AL.

For the entry and exit events of wafer w in restricted equipment region r,

$$0 \le s_{\mathrm{out}(w,r)} - e_{\mathrm{in}(w,r)} \le \overline{R}_{wr}. \tag{25}$$

Equation (25) limits the physical stay in an upper load lock, chamber, lower load lock, or restricted transport region. Material occupancy of a load-lock slot ends when opposite-side pickup is complete. Pressure reset may then overlap subsequent robot travel; the next incompatible load-lock operation must wait until reset is complete.

2.7. Makespan-first two-stage optimization

Let C_w be the time at which product wafer w completes the lower-load-lock return and reaches its load port. Stage one solves

$$C_w = e_{w,\mathrm{return}}, \qquad C_{\max} \ge C_w \ (w \in \mathcal{W}), \qquad \min_{\xi \in \mathcal{F}} C_{\max}. \tag{26}$$

Equation (26) measures makespan at the last product wafer's return to LP after its complete end-to-end route. Internal cleaning affects $C_{\max}$ through resource occupancy and precedence that delay subsequent product returns. Terminal cleaning remains in the action sequence, but its completion time does not directly lower-bound $C_{\max}$. It can still affect other chambers' product returns through competition for the shared VTR; this indirect effect is determined by the complete schedule. WPH uses product-return makespan as its denominator. The joint feasible region $\mathcal{F}$ combines membership, chamber assignment, cycle occupancy, and complete physical timing constraints. Fixed-layout experiments additionally impose their layout and service-order restrictions.

Given stage-one incumbent $\widehat{\xi}$ and makespan $\widehat{C}$, stage two fixes the discrete structure indexed by $\mathcal{D}_{\mathrm{fix}}$ and minimizes linear schedule quality within the makespan tolerance: Here Q^w indexes the evaluated waits. Each ω_α is the nonnegative remainder of its event interval after required processing or movement time is subtracted. L_{sum} sums these waits, including separate waits of different materials that occur concurrently, and $L_{\max}$ bounds the largest item.

For chamber c, cleaning segment e is a production segment between cleanings, including production before the first and after the last cleaning. h_{ceq} denotes the processing start of cycle q within that segment and aliases the corresponding s_o, introducing no independent timing decision. $\mathcal{I}_h$ in the joint model contains three consecutive exposures in the same chamber and 2×2 chain, with a pair retained at both boundaries, $\kappa_{c,q+1} = \kappa_{cq} = 1$; differences do not cross cleaning or chain boundaries. Δ_{ceq} subtracts the preceding start interval from the following one, and $v_{ceq} \ge 0$ bounds its absolute value. Both have units of seconds and measure cadence variation rather than processing duration. The continuous time h_{ceq} differs from binary assignment $h_{o\ell}$; likewise, v_{ceq} differs from pair activation v_p and cycle activation U_{cq}.

$$\begin{aligned}
\xi_d = \widehat{\xi}_d \quad (d \in \mathcal{D}_{\mathrm{fix}}), \qquad & C_{\max} \le \widehat{C} + \varepsilon_C, \\
L_{\mathrm{sum}} = \sum_{\alpha \in Q^w} \omega_\alpha, \qquad & L_{\max} \ge \omega_\alpha, \\
\Delta_{ceq} = (h_{ce,q+1} - h_{ceq}) - (h_{ceq} - h_{ce,q-1}), & \\
v_{ceq} \ge \Delta_{ceq}, \qquad & v_{ceq} \ge -\Delta_{ceq}, \\
\eta_\alpha \ge \omega_\alpha - \bar{\omega}_\alpha, \qquad & \eta_\alpha \ge 0, \\
\min \Phi = \lambda_1 L_{\mathrm{sum}} + \lambda_2 L_{\max} + \lambda_3 \sum_{(c,e,q) \in \mathcal{I}_h} v_{ceq} + \lambda_4 \sum_{\alpha \in Q^w} \eta_\alpha. &
\end{aligned} \tag{27}$$

Equation (27) retains the stage-one feasible makespan bound within the specified tolerance and fixes pairing, chamber and cycle assignments, dummy identities, hand assignments, accompanied-entry choices, and resource orders. Retained same-location atomic synchronization choices and continuous times remain adjustable to reduce total wait, maximum wait, and variation between consecutive processing intervals within each segment. Optional soft-limit excess can also be penalized. The experiments use $(\lambda_1, \lambda_2, \lambda_3) = (1, 10, 25)$ and $\varepsilon_C = 10^{-6}$. With the soft waiting limit disabled, no excess variables are generated. For a time-limited first stage, the best incumbent value $\widehat{C}$ supplies the second-stage bound.

With strictly positive corresponding weights at a stage-two optimum, these upper bounds are tight: $L_{\max} = \max_\alpha \omega_\alpha$, $v_{ceq} = |\Delta_{ceq}|$, and $\eta_\alpha = \max(0, \omega_\alpha - \bar{\omega}_\alpha)$. Auxiliary bounds in a time-limited feasible solution may retain slack, so schedule metrics are recomputed from actual event times. For example, process starts at 100, 200, and 320 s within one segment give intervals of 100 and 120 s, hence $\Delta_{ceq} = 20$ s and a minimum admissible v_{ceq} of 20 s. Starts at 100, 210, and 320 s give zero variation. The cadence term penalizes interval changes, not interval length itself. Soft limits permit penalized excess, unlike hard residency constraints.

3. Structure-aware MILP solution framework

Section 2 links membership and cycle occupancy to complete material routes in a joint model. The solution framework combines structured initialization, MILP search, and timing refinement for both the fixed-configuration special case and the joint model. The fixed-layout benchmark supports training and component comparisons; the joint model evaluates search with the frozen learned policy. Primal feasibility requires coordination of full occupancy, dual-source logistics, and shared-resource order, whereas dual-bound improvement depends on valid inequalities acting on discrete choices, continuous times, and their coupling. We exploit these structures through a structure-preserving binary skeleton (SPBS) for initial feasible scheduling and RDMCT-HBS for cut selection. SPBS supplies a feasible starting point, and RDMCT-HBS determines cut counts and insertion order. SCIP generates valid cuts, checks feasibility, maintains bounds, and certifies optimality [25, 26].

The solution process comprises initial-schedule construction and validation, candidate-cut generation, count prediction,

ordered-subset search, and role-submodular completion, followed by continued branch-and-bound search. All selected cuts are valid inequalities for the original model and retain their coefficients. Cut selection therefore controls the strengthening of the linear relaxation while preserving the original integer feasible set.

3.1. SPBS initial solution

For the evaluated fixed-configuration special case, SPBS retains its pairing, chamber and cleaning layout, constructs a skeleton for selectable resource orders and slots, and then solves for event times in a restricted MILP. Product wafers are paired in input order. The 4×1 pairs are mapped sequentially to chambers, batches, and front/back positions, whereas the 2×2 pairs are assigned round-robin to chamber-chain positions. This determines batch-use, tail-batch, used-position, and last-position indicators. Cleaning counts partition each chamber sequence into process segments: 4×1 batches are assigned to their cleaning epochs, while a 2×2 chain ends its current segment with process-enabling carriers (PECs) when the cleaning threshold is reached and resumes after cleaning with a new PEC-bounded head. Processing assignments, chamber membership, and cleaning boundaries follow from the benchmark configuration; SPBS coordinates dummy occupancy and event times within that layout.

At the resource level, SPBS derives a conservative order from the same physical route. ATR first schedules 4×1 inputs and returns across chambers and stages the first 2×2 pair of a chamber immediately after that chamber's final full batch. It then advances chain inputs and returns segment by segment. Deterministic odd/even load-lock slots break slot symmetry, and the VTR order is aligned with ATR batch progression and cleaning actions. Let $\mathcal{F}_q$ be the binary variables fixed in relaxation level q, with skeleton value $\widehat{x}_j$. The restricted feasibility subproblem imposes

$$\ell_j^{(q)} = u_j^{(q)} = \widehat{x}_j, \qquad j \in \mathcal{F}_q \tag{28}$$

Equation (28) fixes only the skeleton choices at the current level. Start, completion, and waiting times, together with resource orders not fixed at that level, remain free.

SPBS progressively relaxes fixed-variable conditions through five levels, with nested fixed sets $\mathcal{F}_1 \supseteq \mathcal{F}_2 \supseteq \cdots \supseteq \mathcal{F}_5$. For unchanged skeleton values, releasing these conditions expands the restricted feasible set. If a level yields no feasible solution within its allocated budget, the search proceeds to the next level. The first level fixes product and batch assignments, activation states, cleaning segments, load-lock slots, and ATR, AL, LL, and VTR orders. The second releases VTR order; the third also releases load-lock order; the fourth releases all remaining resource orders while retaining structural assignments and slots; and the fifth retains only assignments, activation states, and cleaning segments. For instances with at least 32 wafers, the search starts at the fourth or fifth level. Under a tight waiting-time limit, the original feasible set without additional skeleton restrictions is also considered.

Each level solves a restricted feasibility subproblem with a zero objective and terminates at the first feasible solution. These subproblems use feasibility emphasis, aggressive heuristics, and fast presolve, without cut separation. The resulting principal discrete decisions and continuous event times are then retained in the full model while auxiliary variables associated with schedule stability are completed. A schedule satisfying all original-model constraints becomes the initial incumbent of the main problem. The main problem restores the bounds temporarily tightened by SPBS while retaining the original layout equalities; its feasible set is the complete set conditional on the prescribed layout. In the experiments, SPBS schedules are constructed offline and checked against their corresponding models; the PDI integration interval and main-solve timing both exclude offline initial-schedule construction.

Hierarchical initialization for the joint model starts from a schedule satisfying complete material routes and physical-hand eligibility and improves the skeleton with product makespan as its objective. When enabled, its five levels retain the principal discrete choices, release VTR action orders, release load-lock orders, release the remaining resource orders, and finally retain only structural choices such as pairing, cycle activation, and cycle type. Paired-action synchronization and continuous times remain adjustable at every level. Each level receives at most one fifth of the initialization budget and passes its best complete feasible assignment to the next level. All temporary restrictions are removed before joint main search, which again optimizes pairing, chamber assignment, and resource order. This makespan-improvement initialization differs from the zero-objective feasibility search used for fixed-layout SPBS; construction and main-solve budgets are accounted for separately.

3.2. HEM baseline and role-aware representation

Prior learning-based cut selectors address generic candidate sets with reinforcement learning, learned ranking, and look-ahead imitation [21–23]. HEM further identifies three coupled decisions—which cuts, how many cuts, and in what order—and handles them with a high-level count policy and a low-level sequence policy [24]. We retain HEM's hierarchical decomposition and pointer decoding and use its 13D sequence model as the direct baseline. Training instances, budgets, random seeds, candidate limits, and hidden dimensions are shared while representation, decoding, and post-selection completion are compared. HEM was subsequently extended to hierarchical sequence/set models [27]; HEM-13D here is a controlled reproduction of the 2023 sequence model.

The 13D HEM candidate vector comprises objective parallelism, efficacy, normalized support, integral-support ratio, normalized violation, the mean/maximum/minimum/standard deviation of row coefficients, and the corresponding four objective-coefficient statistics. These features describe immediate LP quality and numerical scale. They do not distinguish two similarly efficacious rows when one acts on discrete route choices and the other on continuous event times. In the present model, that distinction corresponds directly to changing a process/resource combination versus tightening the timing of an existing combination.

Starting from the 13 generic HEM features, variables are partitioned by their mathematical attributes into binary, general

integer, objective-active continuous, continuous auxiliary, and other roles. The coefficient mass of role g in cut i is

$$p_{ig} = \frac{\sum_{j:x_j \in g} |a_{ij}|}{\sum_j |a_{ij}| + \varepsilon}. \tag{29}$$

Equation (29) is the fraction of absolute cut-coefficient mass carried by a mathematical variable role. Beyond the five role masses, the added features are mass on variables with two finite global bounds, positive-coefficient mass, discrete–continuous coupling $4(p_{iB} + p_{iI})(p_{iC_{\text{obj}}} + p_{iC_{\text{aux}}})$, the largest non-residual role mass, and normalized role entropy. The resulting vector has 23 dimensions. Role assignment depends only on variable types, objective coefficients, and global bounds. The representation is therefore invariant to variable renaming that preserves these attributes and the cut coefficients.

HEM uses a hierarchical "count policy + ordered-subset policy" decomposition: its high level predicts the number of cuts from a sequence representation, while its low level uses an LSTM encoder–decoder with pointer attention to select cuts sequentially [24, 28]. Building on this hierarchy, the proposed selector introduces a role-aware grouped gated encoder before the LSTM, a role-submodular completion step after the pointer sequence, and width-three beam search during formal inference, which together form the complete RDMCT-HBS selection pipeline.

3.3. Role-aware encoding and hierarchical sequence decoding

Let m_t be the number of candidates retained at separation round t. For candidate cut i, write the 23D input as $x_i = [x_i^H; x_i^R]$, where $x_i^H \in \mathbb{R}^{13}$ contains the generic HEM features and $x_i^R \in \mathbb{R}^{10}$ contains the proposed role-aware features. The full vector is first layer-normalized; the two feature groups are then projected separately into the common hidden space and fused by an elementwise gate generated from all 23 features:

$$\begin{aligned} \widetilde{x}_i &= \mathrm{LN}(x_i), \\ u_i^H &= W_H \widetilde{x}_i^H + b_H, \qquad u_i^R = W_R \widetilde{x}_i^R + b_R, \\ g_i &= \sigma(W_g \widetilde{x}_i + b_g), \qquad z_i = g_i \odot u_i^H + (1 - g_i) \odot u_i^R. \end{aligned} \tag{30}$$

Here $z_i \in \mathbb{R}^{128}$ is the fused candidate embedding. Unlike a single projection of the 23D vector, the two branches preserve generic LP quality and mathematical-role structure separately, while g_i determines their candidate-specific contribution in each hidden dimension. Thus two cuts with similar efficacy but different concentration on discrete route variables or continuous timing variables can remain distinguishable. This grouped gated adapter is a structural change to HEM's input representation introduced in this work.

The fused candidates, in the deterministic order established by Step 1, are passed through a single-layer LSTM encoder:

$$(h_i, c_i) = \mathrm{LSTM}(z_i, h_{i-1}, c_{i-1}), \qquad i = 1, \ldots, m_t. \tag{31}$$

The states h_i turn local cut descriptors into context-dependent candidate representations and are used by both the high-level retained-count policy and the low-level ordered selector. A 128D hidden state is used to encode the ordered candidates into a contextual sequence representation. Fixing the input order also makes the encoded sequence reproducible for an identical candidate pool.

At low-level decoding step r, the decoder state d_r and all encoder states h_i are used for one glimpse attention followed by pointer attention. Let $M_{ri} = -\infty$ if candidate i already appears in the prefix and $M_{ri} = 0$ otherwise. Then

$$\begin{aligned} a_{ri} &= v_g^\top \tanh(W_e h_i + W_d d_r) + M_{ri}, \\ \alpha_{ri} &= \mathrm{softmax}_i(a_{ri}), \\ \bar{h}_r &= \sum_i \alpha_{ri} h_i, \\ b_{ri} &= v_p^\top \tanh(U_e h_i + U_g \bar{h}_r) + M_{ri}, \\ P(i_r = i \mid i_{1:r-1}, S_t) &= \mathrm{softmax}_i(b_{ri}). \end{aligned} \tag{32}$$

Equation (32) makes the sequential dependence explicit. The selected prefix updates the decoder state and the next conditional distribution. Masking excludes previously selected cuts, and the glimpse summarizes the candidate pool for the current prefix. The resulting sequence model combines contextual candidate encoding, prefix-aware pointer decoding, and the proposed representation and completion stages into one role-aware selector.

Training keeps stochastic sampling from the policy for exploration. During formal inference, instead of committing to one path at every step, the method maintains $B = 3$ partial sequences. For parent beam $b = (i_1, \ldots, i_{r-1})$ and an unselected extension j, the accumulated score is

$$L_r(b \oplus j) = L_{r-1}(b) + \log P(i_r = j \mid b, S_t), \tag{33}$$

All parent-beam–candidate extensions are compared and the three largest L_r values are retained. After the high-level target length k_t is reached, the best sequence is passed to the completion stage. This beam procedure preserves multiple high-probability partial sequences during decoding, reduces early commitment, and works jointly with the subsequent policy-anchored role-submodular completion to inject variable-role structure into the final selected set.

3.4. Stepwise selection from candidate pool to ordered cuts

At separation round t, the method executes the following sequence.

Step 1: form the candidate pool. SCIP first generates and validates candidate rows C_t with its own separators. To control the candidate-state dimension and evaluation cost, at most 128 candidates are retained in descending order of cut efficacy. Ties are broken by original index for deterministic input. Forced cuts remain under solver control and are not placed in the learned ranking.

Step 2: compute and encode the state. The 13 HEM and 10 role-aware features are extracted for every row, together with role masses, dominant role, and structural entropy. Equation (30) then performs grouped gated fusion and Eq. (31) produces contextual candidate states. The encoded candidate set is S_t.

Step 3: determine the retained count. The high-level policy outputs a tanh-bounded action in $[-1, 1]$, which is mapped to a retained fraction ρ_t. The fraction is then clipped by the global 0.2 cap, SCIP's round-specific limit, and the limit of 16 cuts, producing k_t. When enough candidates exist, at least two rows are retained to maintain a nonempty selected subset in each round.

Step 4: construct the candidate order. At low-level step r, the pointer policy conditions on S_t and the selected prefix $i_{1:r-1}$ to score all remaining candidates. Selected indices are masked with negative infinity and cannot be repeated. The joint high- and low-level policy is

$$\pi_\theta(k_t, i_{1:k_t} \mid S_t) = \pi_\theta^H(k_t \mid S_t) \prod_{r=1}^{k_t} \pi_\theta^L(i_r \mid S_t, i_{1:r-1}). \tag{34}$$

Equation (34) means that the high level determines the sequence length and the low level then chooses distinct candidates within that length. Each earlier cut changes both the remaining action set and the decoder state used by later probabilities.

Step 5: beam decoding. The low-level policy samples stochastically during training. At test time, beam search of width three is used. At step r, every partial sequence is extended by every unselected candidate; joint log-probabilities are accumulated, and the three highest-scoring items over all parent-beam–next-candidate pairs are retained. After k_t steps, the best sequence is returned. Unlike one-step greedy decoding, the beam retains an initially lower-ranked prefix whose later joint probability can be higher. This fixed-width neural construction follows the broader use of beam decoding in neural combinatorial optimization [29].

Step 6: role-submodular completion. A sequence based only on joint probability can still repeatedly cover one variable role. The first $\lceil 0.25k_t \rceil$ pointer candidates are retained as policy anchors. The remaining policy order is merged with a heuristic tail sorted by instantaneous structural quality q_i, and at most $2k_t$ rows form expansion pool $\mathcal{R}_t$. For $S \subseteq \mathcal{R}_t$, define

$$\begin{aligned} F_t(S) = &\sum_{i \in S} (0.35 q_i + 0.20 r_i) + 0.25 \sum_{g \in \mathcal{G}} d_g \sqrt{\sum_{i \in S} p_{ig}} \\ &+ \frac{0.20}{|\mathcal{R}_t|} \sum_{u \in \mathcal{R}_t} \max_{i \in S} s_{ui}, \end{aligned} \tag{35}$$

where r_i is the normalized pointer-rank prior, d_g is the demand share of role g in the expansion pool, and s_{ui} is clipped cosine similarity on generic-quality and role-mass descriptors. The score q_i combines efficacy, violation, integral support, recognized-role ratio, dominant-role ratio, and structural entropy. In Eq. (35), the first term retains instantaneous quality and the learned order; the square-root coverage term gives diminishing marginal gain to repeated role mass; and the final term makes the selected cuts represent the main structures in the pool. Starting from the anchors, the method recomputes the marginal gain of every unselected candidate and adds the largest until k_t cuts have been selected. With nonnegative weights, role masses, and clipped similarities, and $F_t(\varnothing) = 0$, the modular, concave-coverage, and facility-location terms form a monotone submodular objective. For fixed anchors A_t and expansion pool, define $f(U) = F_t(A_t \cup U) - F_t(A_t)$. Greedy completion under the cardinality constraint has the standard $1 - 1/e$ approximation property for this residual objective [30]. This ratio concerns coverage under the residual surrogate objective within the fixed pool; PDI and solution time are evaluated experimentally.

The k_t selected cuts are added to the current LP in anchor order followed by greedy-completion order. Candidate truncation, state encoding, count prediction, sequence decoding, and submodular completion jointly determine the ordered cut set used to strengthen the relaxation in the current round.

Algorithm 1: Cut selection by parallel hierarchical REINFORCE and beam search

Algorithm steps

Input: SCIP-validated rows C_t, high-level policy π_θ^H, low-level pointer policy π_θ^L, beam width $B = 3$, and the current SCIP cut limit.
Output: Ordered cut set $S_t^\star = (i_1, \ldots, i_{k_t})$ to be added to the current LP.
1. Train the policies. Two workers collect rollouts in parallel with terminal reward $R = -$PDI; a central learner applies hierarchical REINFORCE with separate EMA baselines and synchronous Adam updates. Fix the parameters for formal inference.
2. Form the pool. Sort by efficacy, break ties by original index, and retain at most 128 rows.
3. Encode the state. Compute 13 HEM features, ten role-aware features, and role masses; apply grouped projections, sigmoid-gated fusion, and LSTM encoding.
4. Determine the count. Predict the retained fraction with π_θ^H and obtain k_t after the 0.2 fraction cap, SCIP round limit, and 16-cut limit.
5. Initialize the beam. Set $\mathcal{B}_0$ to the empty sequence with log probability zero.
6. Expand. For $r = 1, \ldots, k_t$, obtain masked conditional probabilities from one glimpse plus pointer attention, extend every prefix in $\mathcal{B}_{r-1}$ by every unselected candidate, and accumulate log probability.
7. Prune. Retain the three highest joint-probability parent–candidate extensions as $\mathcal{B}_r$; after the loop, take the leading sequence in $\mathcal{B}_{k_t}$.
8. Fix anchors. Retain its first $\lceil 0.25k_t \rceil$ rows as policy anchors A_t.
9. Form the expansion pool. Merge the remaining pointer order with structural-quality order, remove duplicates, and keep at most $2k_t$ rows as $\mathcal{R}_t$.
10. Complete submodularly. Set $S \leftarrow A_t$; while $|S| < k_t$, add the $i \in \mathcal{R}_t \setminus S$ maximizing $F_t(S \cup \{i\}) - F_t(S)$.
11. Add cuts. Add $S_t^\star$ to the current LP in anchor order followed by completion order.

3.5. Policy training and exact solution

One time-limited SCIP solve forms a rollout, with its terminal PDI used as the cost. The current policy is executed in every eligible cut-selection round. Training uses the high-level fraction and low-level sampled sequence at the first valid candidate set of that rollout as its action sample, and the PDI of the complete solve is returned to both policies. Writing $R = -$PDI as a reward, the low-level estimator is

$$\begin{aligned} \widehat{\nabla_\theta J} &= \frac{1}{B} \sum_{b=1}^{B} (R_b - \bar{R}_{\mathrm{EMA}}) \nabla_\theta \log \pi_\theta(\tau_b), \\ \bar{R}_{\mathrm{EMA}} &\leftarrow 0.9 \bar{R}_{\mathrm{EMA}} + 0.1 \bar{R}. \end{aligned} \tag{36}$$

Equation (36) gives the batch policy-gradient update formed from the first eligible candidate-set action sample and an exponential-moving-average baseline; B is the rollout batch and τ_b is the cut sequence sampled at the first eligible candidate set in rollout b. The equivalent loss is PDI-cost advantage times log probability. Low-level log probability is averaged over sequence length so that a longer cut sequence does not produce a larger gradient merely because it contains more terms. The high-level policy uses the same PDI cost and an independent

EMA baseline and is updated every two epochs; both gradient norms are clipped at 2. Parallel environments collect rollouts, and a central learner updates both policies synchronously after each batch, forming a parallel hierarchical REINFORCE procedure [31, 32].

Training uses five instances and five random seeds. Each seed has two workers, each completing four 30-s/1024-MB rollouts per epoch, for eight rollouts per epoch and 60 epochs. Initial Adam learning rates are 10^{-4} for the low-level policy and 5×10^{-4} for the high-level policy and are multiplied by 0.96 every ten epochs. Training uses stochastic decoding and formal inference uses beam width three. Every configuration–seed combination uses the fixed epoch-60 parameters without benchmark- or test-set parameter selection. HEM-13D, feature-only 23D, and the complete 23D method are trained separately so that weights are never mixed across input semantics or completion environments.

RDMCT-HBS only orders and selects valid rows already generated by SCIP, and SPBS submits only solutions checked in the complete model. Neither changes the objective, constraints, integrality, or bound logic. The learned components therefore change the search path and finite-time bound convergence, whereas feasibility and optimality retain the same meaning as in default SCIP.

4. Computational experiments and results

The experiments distinguish a fixed-layout benchmark with homogeneous VTR hands from joint scheduling with clean/dirty-hand eligibility. The former evaluates mode selection and solver components for prescribed layouts; the latter jointly optimizes pairing, chamber assignment, and resource order while assigning physical hands during the solve. Section 4.8 and the subsequent search diagnostics use the clean/dirty-hand model. Earlier experiments use the homogeneous-hand benchmark. Each set is evaluated under its own layout, travel-time, and hand assumptions, to which its numerical conclusions apply.

4.1. Experimental design and metrics

Experiments run on a Windows 11 workstation with an Intel Core Ultra 7 265KF processor, 32 GB RAM, and an NVIDIA GeForce RTX 5060 Ti GPU with 8 GB device memory. Mathematical optimization uses SCIP 10.0.1 and PySCIPOpt 6.1.0; policy training and inference use PyTorch 2.11.0, with Python 3.12.12. The fixed-layout benchmark, policy training, and joint-model validation use the same computing environment. Solve budgets and initialization costs are accounted for separately within each comparison family.

The fixed-layout experiments below evaluate prescribed structures under dual-source supply. Section 4.8 separately evaluates joint-model algorithm comparisons, mode sweeps, source routes, and parameter variations. The experiments examine four questions: how lot size and cleaning interval affect mode preference; whether equal-budget two-stage optimization improves waiting and cadence; how SPBS and structure-aware cut selection separately affect feasible-schedule availability and primal–dual convergence; and how scale, composition, and parameter changes affect computational performance. The evidence comprises mode comparisons, the direct HEM comparison, 960 paired SPBS runs, 1500 equal-budget refinement runs, 300 factorial runs, 80 one-factor perturbation runs, and 729 distribution-shift runs across all methods on 40–64-wafer lots. Four resource Gantt charts show both pure modes for 16 wafers at $H = 10$ and $H = 3$. Every feasible schedule used in the analysis passes SCIP validation against the original model and independent event-time checks.

The compared methods use the same MILP, SCIP version, instances, and random seeds, and each comparison family uses common time and memory limits. Let $P(t)$ denote the primal bound of the minimization problem at time t, namely the objective value of the best incumbent, and let $D(t)$ denote the dual bound, namely the global lower bound certified by the search tree. Define $\Delta(t) = |P(t) - D(t)|$, $m(t) = \min\{|P(t)|, |D(t)|\}$, and $M(t) = \max\{|P(t)|, |D(t)|\}$. When the two bounds are finite, have the same sign, and differ, the terminal relative primal–dual gap is defined in SCIP as

$$G_{\text{end}} = 100\,\frac{\Delta(T)}{m(T)}. \tag{37}$$

SCIP sets $G_{\text{end}} = 0$ when the bounds coincide and returns $+\infty$ when they have opposite signs or valid bounds are not both available. Equation (37) normalizes by the smaller absolute bound and consequently does not cap G_{end} at 100%. A value above 100% indicates that the absolute bound difference exceeds the smaller absolute bound.

SCIP uses a different bounded normalization for the primal–dual integral. Define the instantaneous integral gap as

$$g_{\text{PDI}}(t) = 100\,\frac{\Delta(t)}{M(t)}, \qquad \text{PDI} = \int_0^T g_{\text{PDI}}(t)\,\mathrm{d}t. \tag{38}$$

Equation (38) applies when the bounds are finite, have the same sign, and differ; SCIP sets $g_{\text{PDI}}(t) = 0$ for coincident bounds and $g_{\text{PDI}}(t) = 100$ otherwise. The instantaneous integral gap is therefore bounded to $[0, 100]$; lower PDI means that the bound gap closes earlier. PDI is zero only if the gap is zero for almost the entire timed horizon, whereas $G_{\text{end}} = 0$ only states that optimality is proved at termination. Both quantities follow SCIP definitions and use its solution statistics [25, 26].

The manufacturing instance is the inferential unit: random repeats are first aggregated within an instance, after which paired instance differences use nonparametric 95% bootstrap intervals, Wilcoxon signed-rank tests, and Holm correction within each prespecified analysis family. Time-limited runs remain in the summaries so that random seeds are not treated as independent manufacturing evidence.

The numerical values in Table 3 define the computational configuration. The product-return comparison uses equal cumulative nominal processing time: 180 s for one 4×1 exposure and 90+90 s for two 2×2 exposures. Recipe qualification is a prescribed input condition for this scheduling comparison.

Table 3 specifies the experimental parameters for the representative 16-wafer Gantt charts. Two rotating chambers share

the VTR, while product and dummy wafers compete for robots, load locks, and chamber spaces along their respective routes; cleaning counts, rotation, and transfer times are represented in the same MILP. All displayed schedules satisfy the original-model constraints checked by SCIP and an independent event-time validation. The charts compare resource occupancy and event coordination between the two modes.

4.2. *Process-mode selection driven by cleaning position and lot size*

Steady-state analyses of multi-space CTs usually measure cycle time or throughput [4]. For finite batches, we define $C_{\max}$ as the time when the last product wafer returns to an LP and define equivalent batch output as WPH $= 3600|\mathcal{W}|/C_{\max}$. Pure 4×1 uses 180 s per wafer, pure 2×2 uses 90+90 s, and a 1:1 mixture assigns each product its mode-specific recipe. The cleaning interval H counts full-occupancy cycles per chamber: a 4×1 batch counts once and each 2×2 exposure counts separately. Processing segments follow cycle limits and complete-chain boundaries, including 500-s cleaning and associated dummy loading, unloading, rotation, and transport.

Table 4 compares cleaning endpoints; Table 5 gives the full $H = 3$–10 grid. The $H = 4$–9 experiment contains 36 configurations with training and solver seeds fixed at 1, a 14400-s main-solve limit, and stage 2 disabled. All 36 schedules pass independent validation; 35 are proven optimal. For 16 wafers at $H = 5$, pure 2×2 reaches a feasible 907 s within that budget. A separate 28800-s solve of the same instance, initialized from this schedule, improves the upper bound to 898 s, with lower bound 694 s and terminal gap 29.39%. The improved schedule passes main-model and 4-s exchange checks. This extended-budget experiment determines the feasible upper bound used for mode comparison; its computation time is excluded from the fixed-budget algorithm comparison. With cleaning inactive, the 16-wafer pure 2×2 models at $H = 6$–10 are identical and have optimum 898 s.

For 8 wafers, the three makespans remain 610, 578, and 654 s throughout $H = 3$–10. Pure 2×2 is 5.25% shorter than pure 4×1. At $H = 3$, pure 2×2 and mixed operation generate two and one terminal cleans, respectively; no cleaning occurs at $H \geq 4$. No internal cleaning is required at this lot size, and the ranking remains unchanged across the tested intervals.

For 16 wafers, the preferred mode changes with the interval. At $H = 3$, pure 2×2 and mixed operation each contain two internal cleans, and pure 4×1 is preferred at 944 s. At $H = 4$, pure 2×2 still requires two internal cleans, while mixed operation has only two terminal cleans and improves on pure 4×1 at 939 s. At $H = 5$, pure 2×2 has only two terminal cleans and its feasible 898-s schedule is shorter than the proven optima of both competitors. Thus the preference between the pure modes switches from 4×1 at $H = 4$ to 2×2 at $H = 5$. At $H = 6$–10, all modes avoid cleaning; the pure 2×2 optimum of 898 s is 4.87% and 4.37% shorter than pure 4×1 and mixed operation, respectively.

Complete-chain segmentation and cleaning position jointly explain this change. Each chamber in the 16-wafer pure 2×2 configuration processes four product PWs. The layout splits these into 2+2 PWs at $H = 3$, 3+1 at $H = 4$, and one complete chain at $H \geq 5$. Although $H = 3$ and 4 have equal internal-cleaning counts, their supply and return orders differ; the latter layout yields 1470 s. Mode selection therefore depends on lot size, recipe composition, and admissible complete-chain length. The experiments reconstruct preset complete-chain layouts for each H, so the comparison reflects the joint effect of the cleaning limit and chain layout.

The 24-wafer extension covers all three modes at $H = 3$–6 with 7200 s per configuration. Pure 4×1 has optimum 1268 s at each interval. At $H = 4$, pure 2×2 and mixed-mode lower bounds are 1342 and 1774 s; at $H = 5$, the pure 2×2 lower bound is 1440 s and the mixed optimum is 1796 s. These bounds establish pure 4×1 as preferred at both intervals. At $H = 6$, the mixed-mode upper bound of 1263 s is below the pure 4×1 optimum and pure 2×2 lower bound of 1440 s, establishing mixed-mode dominance; its terminal gap is 10.40%. At $H = 3$, a 28800-s solve for pure 2×2 obtains a validated 2220-s schedule, lower bound 2082 s, and gap 6.63%. This lower bound and the mixed-mode lower bound of 2302 s exceed the pure 4×1 optimum. The extended-budget solve determines mode ordering; its computation time is excluded from the fixed-budget algorithm comparison. Hence pure 4×1 is preferred for 24 wafers at $H = 3$–5 and mixed operation at $H = 6$. The difference from the 16-wafer $H = 5$ result demonstrates the dependence on lot size and complete-chain layout.

All 12 configurations in the 24-wafer $H = 7$–10 extension yield validated schedules. An independent solve for mixed operation at $H = 10$ proves a makespan of 1250 s within the 28800-s budget. The mixed-mode MILPs at $H = 7$–10 have identical objectives, constraints, and variable bounds, so all four intervals have the same optimal makespan. Pure 2×2 yields a feasible 1237-s schedule at $H = 8$–10. Transferring its production-event timing to the $H = 7$ model and scheduling two terminal cleanings also gives a 1237-s schedule that passes validation against the original model. The valid pure 2×2 intervals are $[712, 1237]$ s at $H = 7$ and $[842, 1237]$ s at $H = 8$–10. These upper bounds are below the mixed-mode optimum of 1250 s and pure 4×1 optimum of 1268 s. Thus, pure 2×2 is preferred throughout $H = 7$–10, with the constructed schedules reducing makespan by 1.04% and 2.44%, respectively. Extended-solve and schedule-transfer costs are accounted for separately from the fixed-budget algorithm comparison.

The four resource Gantt charts use 16 wafers and compare the pure modes at $H = 10$ and 3. To study waiting at minimum makespan, the $H = 10$ pure 2×2 schedule is further optimized for total route wait under $C_{\max} \leq 898 + 10^{-6}$ s, with resource order allowed to change. The minimum route wait is 492 s, accumulated over six consecutive-operation intervals for each product wafer: 448 s from AL pair coordination, 36 s before post-processing unloading, and 8 s before return from the lower load lock. This route measure is distinct from the interval between consecutive in-chamber exposures. Each time axis ends at the last product return, $C_{\max}$. Every pure 4×1 process lasts 180 s and every pure 2×2 exposure lasts 90 s.

Chamber lanes distinguish gray exchange actions from pale-pink waiting. A combined exchange by the four-finger VTR

Table 3: Experimental parameters for the cleaning-frequency mode experiment and representative Gantt charts under equal total nominal processing time.

Group	Parameter and value	Group	Parameter and value
Rotating chambers	CH1 and CH2; two shared rotating PMs with four spaces each	Process time	180 s for 4×1; 90+90 s for 2×2
Cleaning	At most $H = 3$–10 effective process cycles between adjacent cleanings; pure-dummy cleaning 500 s	Chamber motion	180° rotation 2 s; inter-action gap 1 s
ATR	One dual-hand robot, capacity 2; load/unload 4 s, complete loaded transfer 11 s	VTR	One four-hand robot, capacity 4; loaded transfer 4 s, empty reposition 4 s
ATR details	LL–LP empty return 6 s; PW-pair exchange at AL 8 s	AL	Capacity 1; minimum alignment occupancy 20 s per wafer
Load locks	LLupper pump/reset 30 s per slot; LLlower vent/reset 25 s	LL capacity	Two independent slots in each of upper and lower LLs
Material source	LP1–LP4, capacity 25 wafers each	Dummy wafers	Eight reusable wafers, four per chamber; storage capacity 10
Batch and model	Product batch size 4; 13 modeled process steps	Chamber bound	At most 15 4×1 batch slots per PM
Model bounds	Big-M = 10000; module and robot residency bounds 10000 s	Waiting option	Avoidable-wait hard cap 0, meaning that this optional cap is disabled
Objective	Product-return makespan and waiting refinement	Policy	Fixed training seed 1, epoch 60, 23 features
Decoder and cuts	Beam width 3; cut pool 128, at most 16 cuts selected per round	Solver	SCIP seed 1; Available, validated initial schedules
Solve and selection	Product-return objective; makespan bounds for mode comparison; waiting-refined schedules for Gantt charts	Schedule validation	Original-model and independent validation for every schedule

Table 4: Cleaning-endpoint comparison under equal cumulative nominal processing time. Cleaning counts sum internal and terminal cleans across both chambers. † marks a validated makespan upper bound; the corresponding WPH uses that makespan.

Lot (wafers)	H	Mode	Total cleans	$C_{\max}$ (s)	WPH (wafers/h)
8	10	Pure 4×1	0	610	47.21
8	10	Pure 2×2	0	**578**	**49.83**
8	10	4:4 mixed	0	654	44.04
8	3	Pure 4×1	0	610	47.21
8	3	Pure 2×2	2	**578**	**49.83**
8	3	4:4 mixed	1	654	44.04
16	10	Pure 4×1	0	944	61.02
16	10	Pure 2×2	0	**898**	**64.14**
16	10	8:8 mixed	0	939	61.34
16	3	Pure 4×1	0	**944**	**61.02**
16	3	Pure 2×2	4	1402	41.08
16	3	8:8 mixed	2	1530	37.65
24	10	Pure 4×1	0	1268	68.14
24	10	Pure 2×2	0	**1237**†	**69.85**
24	10	12:12 mixed	0	1250	69.12
24	3	Pure 4×1	2	**1268**	**68.14**
24	3	Pure 2×2	6	2220†	38.92
24	3	12:12 mixed	4	2324†	37.18

takes 4 s. A split exchange includes 4-s unloading, at least 4 s of empty repositioning, and 4-s loading; other material actions may be coordinated between unloading and loading. In Fig. 6, the first CH1 inter-exposure interval is 63 s: 59 s waiting followed by a 4-s combined exchange. The next product PW, W5/W6, enters the upper load lock at 299 s and completes vacuum-side preparation at 329 s. CH1 finishes processing at 270 s, waits until 329 s, and exchanges at 329–333 s. Most adjacent cycles in the $H = 3$ short-chain schedule use continuous 4-s combined exchanges; CH1 cycles five and six are separated by waits at 1106–1108 s and 1112–1116 s, unloading at 1108–1112 s, and loading at 1116–1120 s. The final exposure starts immediately at 1120 s and ends at 1210 s. Every exposure remains 90 s. Constraining all eight inter-exposure intervals in the same $H = 10$ model to exactly 4 s yields an independently validated optimum of 960 s, 62 s (6.90%) above 898 s. Continuous exchanges compress local intervals but restrict supply and return order on shared resources; allowing local waiting shortens this finite batch's makespan.

4.3. Equal-budget refinement of waiting and cadence

Waiting-time constraints can propagate to manufacturing-system output and control performance [19]. For the same 600-s total budget, the complete two-stage workflow assigns 540 s to the main solve and up to 60 s to refinement, whereas the one-stage control assigns all 600 s to the main solve. Figure 9 shows that the two-stage workflow reduces total route wait, maximum wait, cadence coefficient of variation, and cadence deviation by 51.91%, 68.89%, 39.70%, and 94.05%, respectively. All four paired instance differences remain significant after Holm correction, demonstrating that the two-stage workflow markedly improves waiting and cadence quality under the same total budget.

The quadratic-objective control and the complete linear refinement both use the 540+up-to-60-s split. Linear refinement yields a cadence deviation of 22.04, 63.38% below the quadratic profile's 60.18, and the matched difference remains significant after Holm correction ($p = 0.0377$); total route wait, maximum wait, and cadence coefficient of variation are similar overall. The linear objective therefore reduces the measured

Table 5: Makespans, cleaning counts, and preferred modes under the prescribed PW layouts for 8-, 16-, and 24-wafer lots at $H = 3$–10. Times are in seconds.

Lot (wafers)	H	$C_{\max}$ (s)			$\Delta C_{\max}$ (s)	Total cleans			Preferred mode
		Pure 4 × 1	Pure 2 × 2	1:1 mixed		Pure 4 × 1	Pure 2 × 2	1:1 mixed	
8	3	610	578	654	-32	0	2	1	Pure 2 × 2
8	4	610	578	654	-32	0	0	0	Pure 2 × 2
8	5	610	578	654	-32	0	0	0	Pure 2 × 2
8	6	610	578	654	-32	0	0	0	Pure 2 × 2
8	7	610	578	654	-32	0	0	0	Pure 2 × 2
8	8	610	578	654	-32	0	0	0	Pure 2 × 2
8	9	610	578	654	-32	0	0	0	Pure 2 × 2
8	10	610	578	654	-32	0	0	0	Pure 2 × 2
16	3	944	1402	1530	458	0	4	2	Pure 4 × 1
16	4	944	1470	939	526	0	2	2	1:1 mixed
16	5	944	898†	939	-46	0	2	0	Pure 2 × 2
16	6	944	898	939	-46	0	0	0	Pure 2 × 2
16	7	944	898	939	-46	0	0	0	Pure 2 × 2
16	8	944	898	939	-46	0	0	0	Pure 2 × 2
16	9	944	898	939	-46	0	0	0	Pure 2 × 2
16	10	944	898	939	-46	0	0	0	Pure 2 × 2
24	3	1268	2220†	2324†	952	2	6	4	Pure 4 × 1
24	4	1268	2928†	1827†	1660	0	4	2	Pure 4 × 1
24	5	1268	1769†	1796	501	0	2	2	Pure 4 × 1
24	6	1268	1880†	1263†	612	0	2	1	1:1 mixed
24	7	1268	1237†	1250	-31	0	2	0	Pure 2 × 2
24	8	1268	1237†	1250	-31	0	0	0	Pure 2 × 2
24	9	1268	1237†	1250	-31	0	0	0	Pure 2 × 2
24	10	1268	1237†	1250	-31	0	0	0	Pure 2 × 2

Cleaning counts sum internal and terminal cleans across both chambers. † denotes a feasible upper bound; other times are proven optima. $\Delta C_{\max}$ subtracts the listed pure 4 × 1 time from the pure 2 × 2 time. A preferred mode is established by comparing optima or by a feasible upper bound below the competing lower bounds.

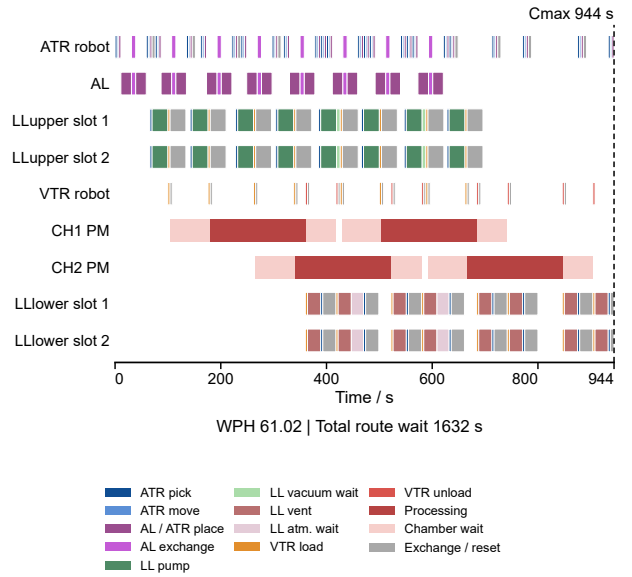


Figure 5: Homogeneous-hand, fixed-layout benchmark: 16-wafer pure 4 × 1 schedule at $H = 10$. No cleaning occurs; $C_{\max} = 944$ s, WPH is 61.02 wafers/h, and total route wait is 1632 s.

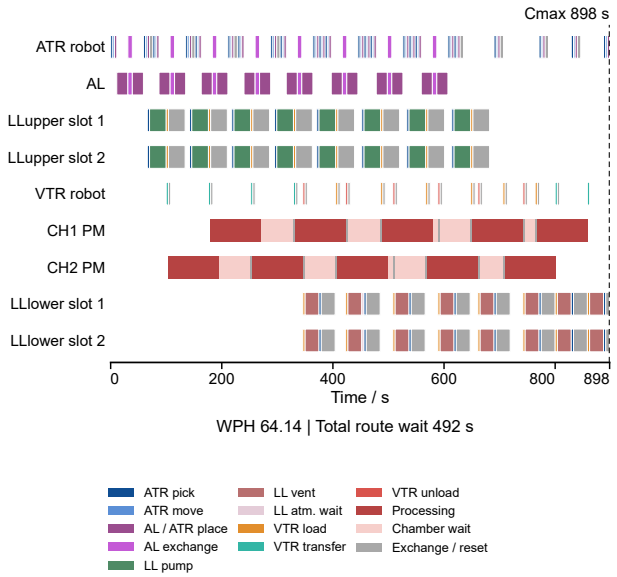


Figure 6: Homogeneous-hand, fixed-layout benchmark: 16-wafer pure 2 × 2 schedule at $H = 10$, without cleaning: $C_{\max} = 898$ s, WPH = 64.14 wafers/h. Refinement minimizes total route wait to 492 s at this makespan. Gray denotes 4-s chamber exchanges; pale pink denotes waiting.

cadence deviation under the same budget.

Table 6 isolates the second stage and refinement objective, complementing the initialization ablation in Table 7.

4.4. SPBS and structure-aware cut selection

Table 7 summarizes the paired SPBS study with 48 instances and ten paired seeds per instance. The timed comparison measures main-solver performance after offline SPBS construction. Relative to the no-start condition, precomputed SPBS increases incumbent availability from 51.67% to 100%, a gain of 48.33 percentage points (95% interval [34.58, 62.29]). Mean PDI falls from 27473.05 to 20278.78, a decrease of 7194.27 or 26.19% (95% interval [5145.78, 9317.61]). Both paired differences remain significant after Holm correction ($p = 2.55 \times 10^{-5}$ and

Table 6: Ablation of second-stage refinement and objective form. Each configuration contains 20 instances and 500 runs under a 600-s total budget. Entries are instance-aggregated means; lower is better.

Configuration	Total wait (s)	Max. wait (s)	Cadence CV	Deviation (s)
Full linear refinement	2596.01	296.33	0.130	22.04
Quadratic refinement	2469.47	293.27	0.134	60.18
Without stage two	5398.60	952.57	0.216	370.41

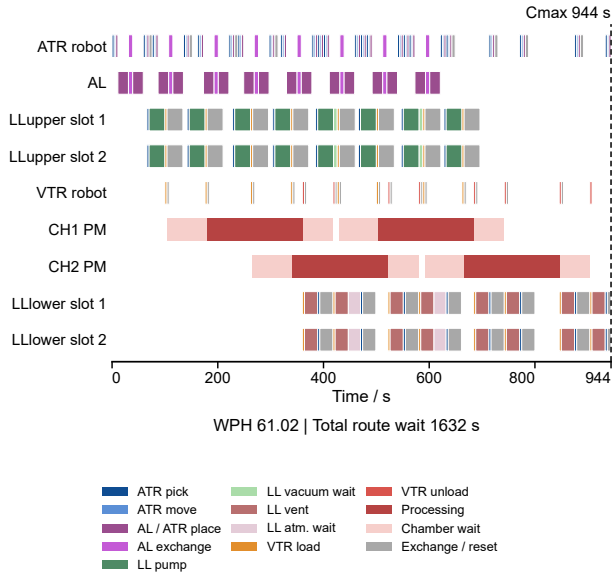


Figure 7: Homogeneous-hand, fixed-layout benchmark: 16-wafer pure 4×1 schedule at $H = 3$: $C_{\max} = 944$ s, WPH is 61.02 wafers/h, and total route wait is 1632 s.

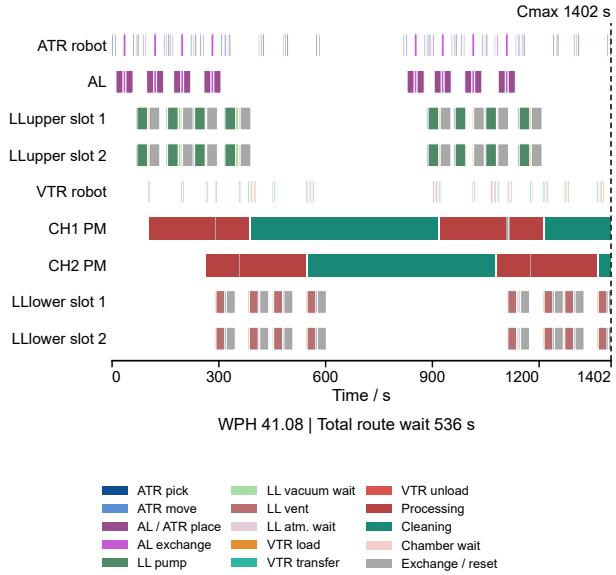


Figure 8: Homogeneous-hand, fixed-layout benchmark: 16-wafer pure 2×2 schedule at $H = 3$: two internal and two terminal cleans, $C_{\max} = 1402$ s, WPH = 41.08 wafers/h, and total route wait = 536 s. The horizon ends at product return; exchanges and waiting are shown separately.

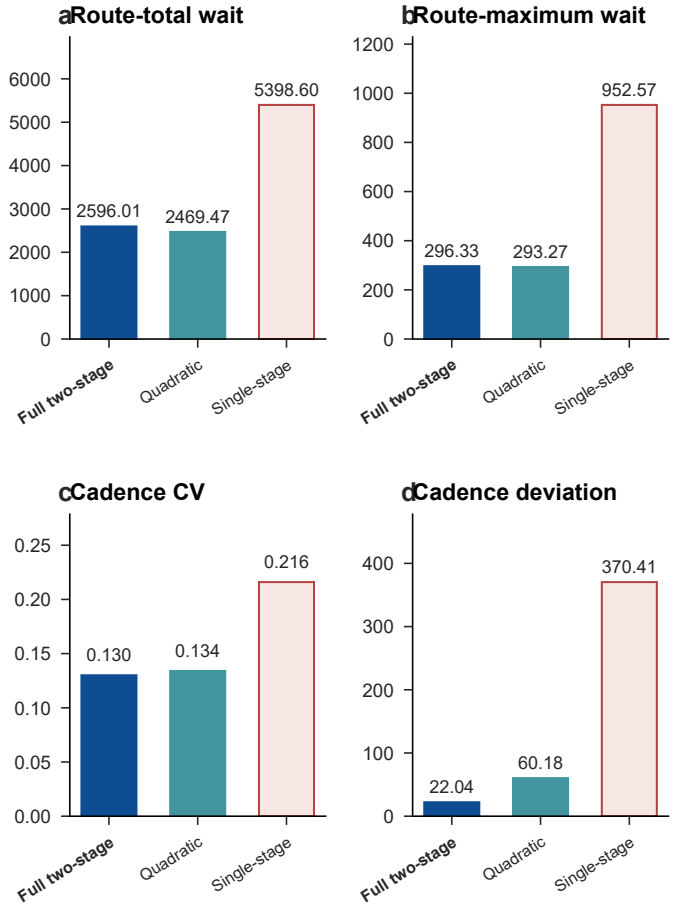


Figure 9: Final-schedule quality under the same 600-s total budget. a–d, total route wait, maximum route wait, cadence coefficient of variation, and cadence deviation; 1500 runs in total, with observed means labeled above the bars.

$p = 3.46 \times 10^{-8}$). Precomputed SPBS therefore improves feasible-schedule availability and primal–dual convergence during the subsequent solve.

Table 7: Effect of precomputed SPBS on time-limited MILP search; construction time is excluded. Each condition contains 48 instances and ten paired seeds per instance, totaling 480 runs.

Metric	No start	SPBS	Observed change
Incumbent rate (%)	51.67	100.00	+48.33 points
Mean PDI	27473.05	20278.78	-26.19%

HEM-13D is the most direct learning baseline for the structure-aware cut selector. Both methods share SPBS, training instances, random seeds, candidate limits, hidden dimensions, and a 600-s solve budget, with 500 runs per method. Table 8 presents this prespecified direct comparison. Relative to

HEM-13D, RDMCT-HBS changes mean PDI from 16823.18 to 16585.70, mean solve time from 219.65 to 218.32 s, mean terminal gap from 127.24% to 126.01%, and proven-optimal rate from 62.4% to 63.2%. All 500 complete-method runs produce feasible schedules. After aggregating repeats over 20 manufacturing instances, all four paired mean-difference 95% bootstrap intervals include zero and all Holm-adjusted p-values are 1.0. The observed means are close under the common SPBS condition; the paired intervals span differences of either sign, and this comparison is therefore expressed in terms of observed means.

Table 8: Direct comparison of RDMCT-HBS and HEM-13D under a common SPBS and budget.

Metric	HEM-13D	RDMCT-HBS	Observed change
Mean PDI	16823.18	16585.70	-1.41%
Mean solution time (s)	219.65	218.32	-0.61%
Mean terminal gap (%)	127.24	126.01	-0.97%
Proven-optimal rate (%)	62.4	63.2	+0.80 points

The comparison jointly evaluates role-aware representation, sequence decoding, and structural completion under common initialization and solver budgets, quantifying their effects on PDI, solution time, terminal gap, and proven-optimal rate with shared SPBS.

4.5. Scale, recipe composition, and computational difficulty

The factorial design contains $4 \times 5 \times 3 = 60$ wafer-count–recipe-composition–process-scale cells with five solver seeds per cell, for 300 runs. Figure 10 shows a sharp transition. All 75 runs at both 8 and 16 wafers prove optimality, with mean PDI values of 28.13 and 1876.54 and mean solution times of 0.34 and 33.86 s. At 24 and 32 wafers, the proven-optimal rates fall to 36% and 20%, mean PDI rises to 31155.94 and 32959.89, and mean solution time reaches 408.55 and 433.20 s.

Table 9 summarizes the algebraic dimensions of the factorial instances. Each wafer count comprises 15 distinct parameter configurations; scalar variables and constraints are counted before presolve, without duplicating models across solver seeds. The mean variable counts are 2645.2 and 6711.2 for 16 and 24 wafers, respectively. Under the nominal process scale and the 1:1 mixed recipe, the corresponding counts are 2722 and 6630. These totals include layout-fixed indicators and linearization auxiliaries and thus characterize the algebraic representation; independent decisions also depend on fixing relations and process constraints.

Algebraic representation size and the number of candidate sequences follow different growth rates. For K_r candidate operations on resource r, pairwise ordering requires at most $\binom{K_r}{2}$ binary variables, giving an upper bound of $\sum_{r\in\mathcal{R}} \binom{K_r}{2}$ across resources. Before precedence restrictions are imposed, a single resource admits $K_r!$ permutations. Process precedence and resource coupling further restrict these candidate orders. Thus, a quadratic ordering representation can encode factorially many candidate sequences, and variable dimension and combinatorial search size should be assessed separately.

The prespecified PDI factorial model has $R^2 = 0.998$. Under the 1:1 mixed recipe, the additional interaction coefficients at 24 and 32 wafers are 37729.48 and 40788.63, while the 95% intervals for the process-scale main effect and all prespecified process-scale interactions include zero. Over the tested 0.9–1.1 process-scale range, observed differences in computational difficulty are primarily associated with the interaction between batch size and mixed-flow composition, with comparatively small differences associated with process scale.

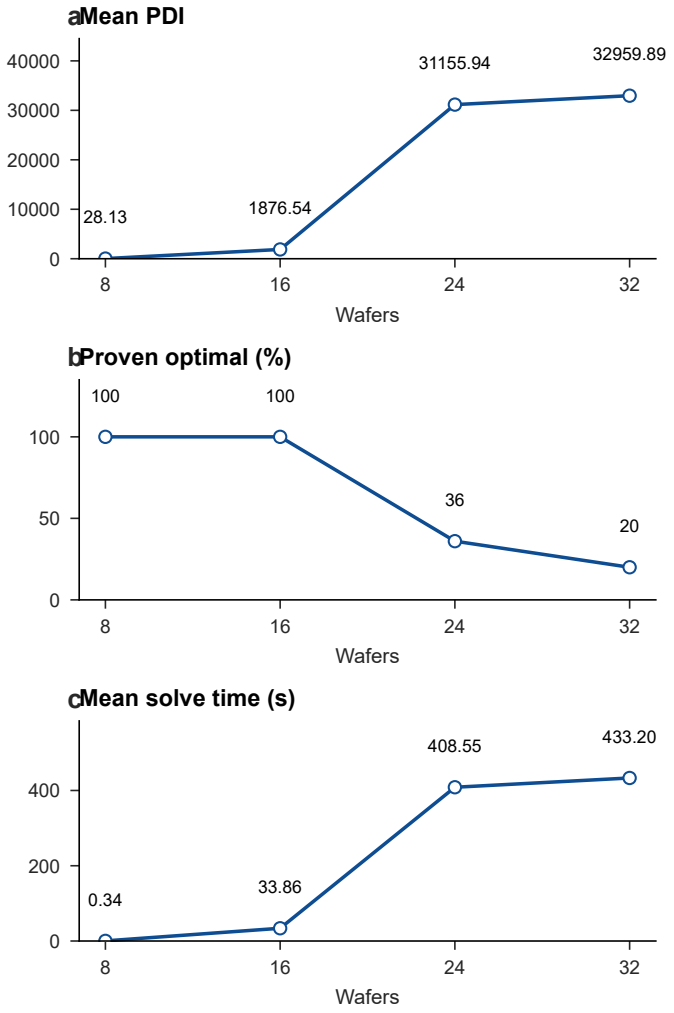


Figure 10: Scale transition in the 300-run factorial experiment. a, mean PDI; b, proven-optimal rate; c, mean solution time. Each wafer count contains 75 runs and observed values are labeled directly.

4.6. Operation under parameter perturbations

Prior CT studies show that cleaning actions and their triggering conditions alter robot-task sequences, cycle time, and wafer delay [14, 16]. The one-factor sensitivity study therefore covers eight prescribed perturbations of transport, processing, cleaning interval, and vacuum-domain dummy inventory, for 80 runs. Every run returns a solver-checked feasible schedule and 20 runs prove optimality; scenario mean PDI ranges from 28899.41 to 42478.32. A longer cleaning interval gives the lowest mean PDI and slower transport the highest, describing the variation in search performance across the tested parameter settings.

Table 9: Model dimensions in the factorial experiment with prescribed PW layouts. Each wafer count comprises 15 models. Entries are means except for the variable range; all models have zero general integer variables.

Wafers	Variables	Binary	Continuous	Variable range	Constraints
8	1163.6	644.6	519.0	892–1450	2947.0
16	2645.2	1632.4	1012.8	2217–3457	8727.0
24	6711.2	4962.4	1748.8	5507–9362	23670.8
32	9984.0	7547.2	2436.8	7801–13663	41630.8

4.7. Feasible-schedule delivery under distribution shift

The distribution-shift suite enlarges batches to 40–64 wafers and changes mixed-flow composition, producing 729 runs across all compared methods over nine shifted manufacturing scenarios, including 135 runs of the complete method. This experiment evaluates schedule availability and search performance under changes in scale and composition. All 729 runs return an incumbent that passes SCIP and independent post-solve checks. The shared SPBS–MILP–SCIP chain therefore continues to output a validated resource-action schedule over the tested changes in scale and recipe composition. The proven-optimal rate is 0%. The complete method has mean PDI 73919.15, compared with 73670.99 for default SCIP and 70915.54 for adaptive cut scoring. Feasible-schedule availability is 100% across this suite, while adaptive cut scoring has the lowest observed mean PDI.

4.8. Joint scheduling with clean and dirty VTR hands

Six supplementary configurations cover single- and dual-source six-wafer lots at $H = 3$, dual-source eight-wafer lots at $H = 3$ and 10, and dual-source 16- and 24-wafer lots at $H = 3$. Single-source dummies travel between atmospheric storage and the chambers via ATR and the load locks without alignment; dual-source dummies use vacuum storage and VTR. Each lot contains equal numbers of the two product recipes, distributed successively across LP1–LP4; same-recipe products may be paired across load ports. Eight physical dummies form a shared inventory. VTR pickup, travel, and placement take 1, 2, and 1 s, respectively, and rotation takes 2 s. No additional travel is imposed between upper and lower slots at the same load-lock interface. Processing takes 180 s or two 90-s exposures. The fixed policy assigns same-recipe pairs to both chambers according to nominal workload; the joint model also selects membership, chamber assignment, and cycle occupancy. Both formulations optimize resource orders and event times. The VTR uses two dirty hands for unprocessed products and two clean hands for fully processed products; dummies may use any free hand. Each complete holding interval occupies one unchanged physical hand.

All five settings within each configuration use exactly the same initial variable assignment, with solver and policy-training seeds fixed at 1. Initial construction and auxiliary-variable completion are timed separately. The fixed policy, default SCIP, HEM, and the complete selector each receive a 300-s main-solve budget; refinement receives 270 s for the main solve and up to 30 s for the second stage. All six refinement configurations use default SCIP and are compared with its 300-s one-stage results. Learned policies retain the frozen epoch-60 parameters trained on the fixed-layout benchmark. All 30 schedules pass independent checks of complete material routes, exposure counts, resource capacities, cleaning, and dummy reuse. Table 10 gives feasible makespans under common initialization, computed as the actual time of the final product return to its load port.

The fixed policy obtains shorter feasible makespans in four six- or eight-wafer configurations and ties the joint methods for 16 and 24 wafers. All 18 joint primary runs with default SCIP, HEM, and the complete selector retain the common initial pairing, dummy identities, chamber assignments, and cycle structure; product makespans also equal their initial values. Initialization supplies the feasible structures in this comparison, and the subsequent timing-refinement gains are given in Table 13. Learned selection is called once in each six- and eight-wafer configuration and zero times in the 16- and 24-wafer configurations. All three joint methods return the same structures and makespans within the tested budget. Differences between fixed and joint settings describe time-limited search performance; the joint feasible region contains the corresponding fixed-structure subset.

Four- and six-product dual-source mixed instances further examine the relationship between fixed-structure and joint search. The two recipes have product counts of 2:2 and 3:3, respectively; both instances use $H = 10$, eight shared dummies, and full candidate bounds. The four-product instance places one 4×1 product at each of LP1 and LP2 and one 2×2 product at each of LP3 and LP4. The six-product instance places one product of each recipe at both LP1 and LP2, with one additional 4×1 product at LP3 and one 2×2 product at LP4. The two settings for each instance share a complete initial variable assignment, use default SCIP with seed 1, and allow up to 1800 s for the primary solve with no second stage. Initialization takes 0.296 and 0.781 s, respectively, outside the main-solve budget. Table 11 gives both terminal lower bounds and feasible upper bounds. All four schedules pass complete-route, individual-hand, and dummy-reuse checks.

The four-product fixed structure has an optimal makespan of 532 s, 40.09% below the common initial schedule. The six-product fixed structure yields a feasible makespan of 957 s, a reduction of 37.45%. Both joint solves retain the initial pairing, dummy identities, chamber assignments, and cycle occupancy; the four-product improvement from 888 to 717 s arises from timing and resource coordination within that structure. The complete assignments for the 532- and 957-s fixed-structure

Table 10: Equal-budget comparison of joint scheduling and a fixed policy with clean/dirty VTR hands. Initial gives the common starting makespan; Fixed uses fixed-policy SCIP. The other methods use the joint model, with 300 s per primary solve. Times are in seconds.

N	Source	H	Initial	Fixed	SCIP	HEM	Proposed
6	Dual	3	2051	1903	2051	2051	2051
6	Single	3	3917	3718	3917	3917	3917
8	Dual	3	2300	2075	2300	2300	2300
8	Dual	10	1779	1235	1779	1779	1779
16	Dual	3	4221	4221	4221	4221	4221
24	Dual	3	6823	6823	6823	6823	6823

Table 11: Small-instance structural comparison for dual-source mixed lots ($H = 10$; times in seconds). Both settings allow up to 1800 s per primary solve; the four-product fixed structure closes the bound gap in 45 s.

N	Initial	Fixed structure		Joint model	
		Lower	Upper	Lower	Upper
4	888	532	532	46	717
6	1530	725	957	39	1530

schedules also satisfy the unrestricted joint model. These schedules therefore supply valid feasible upper bounds for the joint problems, while Table 11 preserves the bounds obtained by each independent search. The joint solves process 12 and 3 nodes, respectively, indicating slow search progress for this formulation after structural decisions are released within the tested budget.

The selected structures in both instances repeatedly borrow the same dummies across the two chambers: every productive cycle contains D1 and D2. A physical dummy must be returned before its next loan, forcing the corresponding chamber exposures to occur sequentially. Balanced nominal processing loads therefore do not automatically produce chamber concurrency; dummy identities and complete loan intervals constrain the achievable overlap. None of these four schedules includes cleaning, so this comparison examines the coupling among dummy reuse, clean/dirty-hand allocation, and transport timing. Total dummy inventory, actual loan assignments, and the resulting event times must consequently be evaluated together.

Further structural checks initialize the complete joint model with the 532- and 957-s fixed-structure schedules, retaining all pairing, dummy-identity, chamber-assignment, and cycle decisions. Each run receives an additional 1800 s with seed 1 and no second stage. Both retain the supplied principal structure and makespan, with terminal lower bounds of 46 and 39 s, respectively. Table 12 presents these strong-initial-solution checks and a controlled dummy-identity comparison. Additional search time is accounted for separately from the preceding solves that produce the initial schedules.

The dummy-identity comparison preserves the four-product pairing, chamber assignments, cycle types, occupancy, and front/back structure, using the same shared inventory of eight dummies. CH1 uses D1/D2 and CH2 uses D3/D4. This comparison fixes the revised dummy-loan structure while optimizing resource order, physical-hand assignments, and event times. Starting from a 628-s schedule obtained by a 60-s preliminary search, an additional 1800-s primary solve yields a feasible makespan of 403 s and a terminal lower bound of 393 s. The complete assignment also satisfies the unrestricted joint model, supplying a feasible upper bound of 403 s for the four-product joint problem.

In the revised schedule, CH1 processes in 4×1 mode during 173–353 s, while CH2 executes its two 2×2 exposures during 104–194 s and 204–294 s, producing 111 s of concurrent processing. The four products return to their original LPs at 403, 396, 347, and 340 s. Complete routes, exposure counts, individual-hand occupancy, and dummy reuse satisfy all constraints; no cleaning is required. The 403-s makespan is 129 s (24.25%) below the proven optimum of 532 s under the original dummy-loan structure. Because every timing arrangement within that original structure is bounded below by 532 s, the strict improvement establishes that revised dummy identities combined with timing reoptimization can outperform all schedules retaining the original structure. The comparison quantifies the value of dummy-loan structure at unchanged inventory. The common-initial-solution algorithm comparison remains as given in Table 10.

Refinement sums transport-route waiting for both products and dummies, subtracting required travel, alignment, or pressure-transition durations from successive handling and supply/return intervals. This measure is evaluated separately from the product-route waiting used in the preceding fixed-layout experiments. Table 13 shows reductions of 8.40%–29.21% in total transport-route waiting across the six configurations, with maximum individual waiting decreasing from 73 to 65 s in every case. For the six-wafer single-source configuration, total waiting decreases from 2200 to 1680 s, a reduction of 23.64%, and actual product makespan decreases from 3917 to 3875 s. Actual makespans also decrease in all five dual-source configurations. Under the common 300-s total solve budget, continuous-time refinement thus reduces both transport waiting and product makespan in the tested configurations.

The dual-source joint-model sweep covers $N = 8, 16, 24$ and $H = 3, \ldots, 10$, with pure 4×1, pure 2×2, and an equal-product-count mixture at each configuration. Each cell receives up to 20 s of hierarchical initialization and a 300-s main solve. Table 14 gives 72 independently validated feasible makespans, comparing schedules obtained under this budget and initialization strategy.

Among the schedules obtained, pure 2×2 gives the shortest eight-wafer makespan at $H = 5, 7$, whereas mixed operation gives the shortest at $H = 8$–10. For 16 wafers, the smallest observed makespans come from pure 4×1 at $H = 3$–6, mixed operation at $H = 7, 8$, and pure 2×2 at $H = 9, 10$. For 24 wafers, pure 4×1 gives the shortest makespan at every tested H, decreasing from 4640 s at $H = 3$ to 3945 s at $H = 6$–10, a reduction of 14.98%. These differences characterize the combined effects of cleaning conditions, lot size, and finite-budget search on the schedules obtained.

To examine both source routes, physical hands are assigned to eight existing schedules with two or four mixed-recipe products and $H = 2, 3$. The original schedules used homogeneous hands, a 120-s main-solve budget, and up to 20 s of hierarchical initialization. All eight satisfy clean/dirty-hand eligibility

Table 12: Strong-initial-solution and dummy-identity checks ($H = 10$, dual-source; times in seconds). Additional solves receive 1800 s each; the original fixed result is from Table 11. The final-row lower bound applies to the revised fixed dummy structure. Overlap is the total intersection of chamber-processing intervals. Preceding construction and solves are timed separately.

N	Setting	Initial	Lower	Upper	Overlap
4	Original fixed	888	532	532	0
4	Joint, strong start	532	46	532	0
6	Joint, strong start	957	39	957	0
4	Separate dummy identities	628	393	403	111

Table 13: Equal-total-budget transport-wait refinement with clean/dirty VTR hands (s). Superscripts (1)/(2) denote 300-s one-stage and 270+up to 30-s two-stage solving. All rows use default SCIP. Makespans use actual product return times.

N	Source	H	$L^{(1)}_{\text{sum}}$	$L^{(2)}_{\text{sum}}$	Gain (%)	$C^{(1)}_{\max} \to C^{(2)}_{\max}$
6	Dual	3	356	252	29.21	2051 → 1916.91
6	Single	3	2200	1680	23.64	3917 → 3875
8	Dual	3	532	480	9.77	2300 → 2150.96
8	Dual	10	524	480	8.40	1779 → 1732.96
16	Dual	3	1100	960	12.73	4221 → 3886.80
24	Dual	3	1624	1440	11.33	6823 → 6066.30

Table 14: Feasible makespans for the clean/dirty-hand joint model under a 300-s main-solve budget (s). Mixed operation uses equal product counts of both recipes.

N	H	4×1	2×2	Mixed
8	3	1315	1658	1934
8	4	1315	1798	1315
8	5	1315	1230	1264
8	6	1315	1318	1315
8	7	1315	1267	1315
8	8	1315	1267	1227
8	9	1315	1267	1227
8	10	1315	1281	1227
16	3	2975	4716	3423
16	4	2633	3470	3056
16	5	2633	3056	2955
16	6	2633	2976	3055
16	7	2633	2874	2599
16	8	2633	3064	2585
16	9	2633	2554	2585
16	10	2633	2591	2636
24	3	4640	7336	5991
24	4	4361	6060	5359
24	5	4358	5434	5473
24	6	3945	5434	4835
24	7	3945	4796	4835
24	8	3945	4799	4835
24	9	3945	4799	4838
24	10	3945	4796	4203

without changes to event times. Table 15 gives the recertified makespans. The two-product instances use different recipes and pair each product with a dummy; the four-product instances pair same-recipe products across LP1 and LP2 and across LP3 and LP4. Products pass through AL and the load locks. Single-source dummies travel directly from atmospheric storage to LL through ATR, whereas dual-source dummies use vacuum supply and return. All complete routes, exposure counts, and shared-inventory constraints are satisfied. Dual-source schedules have shorter feasible makespans in all four configurations.

Processing-time and dummy-inventory variations are further evaluated with eight mixed-recipe products, $H = 3$, and dual-source supply, using 300 s for the main solve and 20 s for initialization. Table 16 covers ±10% changes in nominal processing

Table 15: Makespans of small single- and dual-source schedules recertified with clean/dirty hands (s). Single-source dummies bypass alignment.

N	H	Single source	Dual source
2	2	2378	1028
2	3	1433	514
4	2	2436	1280
4	3	1491	854

times and inventories of four, six, and ten dummies. All five cells yield complete feasible schedules, establishing schedule availability under these input changes for the modeled pairing, cleaning, and transport relations. The makespans are obtained by separate searches for the respective inputs.

Table 17 gives formulation dimensions before presolve for the two pure modes with 16 products and $H = 10$. Physical-hand allocation introduces both hand selection and exclusion between holding intervals on the same hand: the respective hand-assignment counts are 672 and 992, and hand-order counts are 14028 and 30628. Chamber order is represented through cycle occupancy, first/last exposure, and front/back states. These counts include activation conditions, symmetric choices, and precedences implied by routes and thus measure algebraic representation size; feasible combinations are further restricted by material, capacity, and timing constraints.

For $H = 10$, dual-source supply, eight dummies, and equal recipe counts, the complete 16- and 24-wafer models contain 124048 and 296700 variables, respectively. Considering only product pairing and assignment to two labeled chambers, even recipe counts give $N_\zeta! N_\psi! / [(N_\zeta/2)!(N_\psi/2)!]$ candidates: 2822400 for 16 wafers and 442597478400 for 24. This count includes label symmetry and precedes full-occupancy, cleaning, and routing constraints. It measures growth of a specified candidate class, rather than feasible schedules or independent variable dimensions.

4.9. Search-stage diagnostics with strengthened bounds

For the dual-source mixed configurations with 8 and 16 products and $H = 3$, default SCIP and the complete selector are each run for 120 s with a common feasible initial schedule and identical strengthened bounds. These comprise analytical lower bounds from mandatory routes and resource occupancy and a makespan upper bound from the initial schedule. The four diagnostic runs are separate from the preceding 300-s comparisons and identify the stages consuming the search budget.

For eight products, both methods retain a feasible makespan of 2300 s and terminate with a lower bound of 492 s. Both enter the first root LP solve, executing 25630 and 24675 LP iterations, respectively, before any learned cut-selection call. For 16 products, both retain a feasible makespan of 4221 s and spend the 120-s budget in presolve, with zero processed nodes and zero LP solves. All four final schedules pass independent physical validation. These observations locate the computational effort in presolve and root relaxation, identifying the large discrete formulation and continuous-relaxation solution as the initial search costs for these joint instances.

Table 16: Parameter-variation results for the clean/dirty-hand joint model. Times are in seconds; processing durations refer to one exposure in each mode.

4×1 duration	2×2 duration	Dummies	$C_{\max}$
162	81	8	1880
198	99	8	1988
180	90	4	1875
180	90	6	1888
180	90	10	1937

Table 17: Size audit of the 16-product, $H = 10$ joint models; counts refer to candidates before presolve.

Variable family	4×1	2×2
ATR order	4560	4560
VTR action order	27730	63190
VTR hand assignment	672	992
VTR hand-occupancy order	14028	30628
AL order	120	120
Upper-LL order	120	120
Lower-LL order	120	120
Dummy-loan order	2244	5724
All variables	78351	180925
All constraints	376764	887126

4.10. Discussion of mode selection and computational performance

The clean/dirty-hand joint model coordinates product pairing, dummy allocation, chamber occupancy, and physical-hand assignment. All 107 main runs and seven small-instance structural checks produce complete executable schedules. Resource-order and timing optimization within the original fixed structures improves the four- and six-product makespans from 888 and 1530 s to 532 and 957 s, respectively. Joint search initialized with these schedules retains their principal structures within the tested budget. The dummy-identity comparison instead reduces the four-product makespan to 403 s using the same inventory of eight dummies, with 111 s of concurrent chamber processing. Even with identical product pairing, chamber assignments, and cycle structure, dummy identities can therefore alter attainable concurrency and minimum makespan. Structural selection must account for both product organization and dummy-loan relationships. Equal-budget refinement further quantifies waiting and makespan improvements within a given discrete structure.

Mode comparisons depend on equipment assumptions and solution settings. In the homogeneous-hand fixed-layout benchmark, internal cleaning delays subsequent products, with mixed operation preferred for 16 wafers at $H = 4$ and 24 wafers at $H = 6$. For 24 wafers at $H = 7$–10 in that benchmark, the pure 2×2 feasible upper bound of 1237 s lies below both competing optima. The clean/dirty-hand joint model compares schedules under a different set of travel times, hand restrictions, and a 300-s search budget; pure 4×1 gives the shortest observed makespan for 24 wafers at every tested cleaning interval. These results respectively support mode selection within prescribed layouts and evaluation of schedules obtained by joint search.

Solver components affect different performance measures. Precomputed SPBS improves feasible-schedule availability during time-limited fixed-layout search, and two-stage refinement reduces waiting and improves cadence. With common SPBS, the complete selector and HEM have close observed mean metrics. The clean/dirty-hand experiments further show that initialization supplies an executable starting point for joint scheduling, timing refinement improves the selected structure, and learned cut selection acts only once separation is reached. Feasible-schedule availability, timing quality, and bound convergence therefore require separate evaluation.

5. Conclusions

This study develops a two-stage joint scheduling model for PECVD rotating-chamber cluster tools. Wafer pairing connects product and dummy supply/return routes, while chamber assignment, resource order, physical-hand assignment, and event times jointly represent full occupancy, dummy reuse, periodic cleaning, and residency constraints. Single- and dual-source configurations differ in the origin and route of dummy wafers, and clean/dirty-hand eligibility applies throughout each holding interval.

All 107 main runs and seven small-instance structural checks of the clean/dirty-hand model produce independently validated physical schedules. For four products, reassigning dummy identities and reoptimizing timing while preserving product pairing, chamber assignments, cycle structure, and the eight-dummy inventory reduces makespan from the original fixed-structure optimum of 532 s to 403 s, a reduction of 24.25%, with 111 s of concurrent chamber processing. Dummy-loan relationships are thus substantive structural decisions affecting attainable concurrency and makespan. Across six single- and dual-source configurations under the same 300-s total solve budget, two-stage refinement reduces total transport-route waiting by 8.40%–29.21%, decreases maximum individual waiting from 73 to 65 s, and shortens actual product makespan in every case. The 72 mode-sweep cells for lots of 8, 16, and 24 wafers characterize feasible schedules under each cleaning condition; pure 4×1 gives the shortest observed makespan for 24 wafers throughout the tested $H = 3$–10 range. Eight small source-route schedules additionally realize product–dummy pairing, cross-LP pairing of same-recipe products, and both supply/return routes with valid clean/dirty-hand assignments.

The homogeneous-hand fixed-layout benchmark separately quantifies mode selection and solver components. Under equal cumulative nominal processing time, lot size and cleaning position jointly change mode preference. For 16 wafers without cleaning, the pure 2×2 makespan is 898 s, 4.87% below pure 4×1. Under the same 600-s total budget, two-stage refinement reduces total route waiting by 51.91%. Precomputed SPBS raises feasible-schedule availability in the subsequent main solve from 51.67% to 100% and reduces mean PDI by 26.19%. Together with the clean/dirty-hand experiments, these results support joint coordination of dummy supply/return, cleaning, and product processing, with structured initialization and continuous-time refinement contributing to schedule construction and operating quality, respectively.

CRediT authorship contribution statement

Lincong Wei was responsible for Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, and Writing – review and editing. Lizhen Shao was responsible for Conceptualization, Methodology, Formal analysis, Validation, Writing – review and editing, and Project administration. Wei Wang was responsible for Formal analysis, Investigation, Writing – review and editing, and Supervision. All authors reviewed and approved the final manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and research-material availability

The data, mathematical instances, trained models, and solver software supporting the findings are available from the corresponding author on reasonable request.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI ChatGPT and Codex to assist with language editing, Chinese–English translation, LaTeX consistency checks, the development and debugging of numerical solution methods, and experimental visualization. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] Pan, C.R., Zhou, M.C., Qiao, Y., Wu, N.Q., 2018. Scheduling cluster tools in semiconductor manufacturing: Recent advances and challenges. IEEE Transactions on Automation Science and Engineering 15, 2, 586–601. doi:10.1109/TASE.2016.2642997.

[2] Applied Materials, n.d. Producer XP Precision CVD. https://www.appliedmaterials.com/us/en/product-library/producer-xp-precision-cvd.html (accessed 7 September 2026).

[3] Gu, L., Wu, N., Qiao, Y., Zhang, S., Li, T., 2024. Scheduling cluster tools with multi-space process modules and a multi-finger-arm robot in wafer fabrication subject to wafer residency time constraints. Applied Sciences 14, 9490. doi:10.3390/app14209490.

[4] Gu, L., Wu, N., Qiao, Y., Zhang, S., 2024. Scheduling of Multi-finger-robotic Cluster Tools with Multi-space Process Modules, in: 2024 International Conference on Networking, Sensing and Control (ICNSC), IEEE. doi:10.1109/ICNSC62968.2024.10760109.

[5] Perkinson, T.L., McLarty, P.K., Gyurcsik, R.S., Cavin, R.K., 1994. Single-wafer cluster tool performance: An analysis of throughput. IEEE Transactions on Semiconductor Manufacturing 7, 369–373. doi:10.1109/66.311340.

[6] Rostami, S., Hamidzadeh, B., 2002. Optimal scheduling techniques for cluster tools with process-module and transport-module residency constraints. IEEE Transactions on Semiconductor Manufacturing 15, 341–349. doi:10.1109/TSM.2002.801379.

[7] Wu, N.Q., Zhou, M.C., 2010. A closed-form solution for schedulability and optimal scheduling of dual-arm cluster tools with wafer residency time constraint based on steady schedule analysis. IEEE Transactions on Automation Science and Engineering 7, 303–315. doi:10.1109/TASE.2008.2008633.

[8] Qiao, Y., Wu, N.Q., Zhou, M.C., 2012. Petri net modeling and wafer sojourn time analysis of single-arm cluster tools with residency time constraints and activity time variation. IEEE Transactions on Semiconductor Manufacturing 25, 432–446. doi:10.1109/TSM.2012.2199338.

[9] Qiao, Y., Wu, N., Zhou, M.C., 2014. Scheduling of dual-arm cluster tools with wafer revisiting and residency time constraints. IEEE Transactions on Industrial Informatics 10, 286–300. doi:10.1109/TII.2013.2272702.

[10] Wang, J., Hu, H., Pan, C., Zhou, Y., Li, L., 2020. Scheduling dual-arm cluster tools with multiple wafer types and residency time constraints. IEEE/CAA Journal of Automatica Sinica 7, 776–789. doi:10.1109/JAS.2020.1003150.

[11] Wang, J., Liu, C., Zhou, M.C., Abusorrah, A., 2025. Virtual cell-based scheduling approach to single-robotic-arm cluster tools subject to wafer residency time constraints. IEEE Transactions on Automation Science and Engineering 22, 240–251. doi:10.1109/TASE.2024.3349380.

[12] Yang, F., Li, C., Zhen, L., Zhang, C., Wu, N., 2025. Scheduling residency time-constrained single-armed cluster tools with two-wafer types via mixed integer programming model. Expert Systems with Applications 288, 128209. doi:10.1016/j.eswa.2025.128209.

[13] Kim, H., Kim, H.J., Lee, J.H., Lee, T.E., 2013. Scheduling dual-armed cluster tools with cleaning processes. International Journal of Production Research 51, 3671–3687. doi:10.1080/00207543.2012.758392.

[14] Yu, T.S., Lee, T.E., 2020. Wafer delay analysis and control of dual-armed cluster tools with chamber cleaning operations. International Journal of Production Research 58, 434–447. doi:10.1080/00207543.2019.1593547.

[15] Qiao, Y., Lu, Y., Li, J., Zhang, S., Wu, N., Liu, B., 2022. An efficient binary integer programming model for residency time-constrained cluster tools with chamber cleaning requirements. IEEE Transactions on Automation Science and Engineering 19, 1757–1771. doi:10.1109/TASE.2021.3122576.

[16] Li, C., Yang, F., Zhen, L., 2022. Efficient scheduling approaches to time-constrained single-armed cluster tools with condition-based chamber cleaning operations. International Journal of Production Research 60, 3555–3568. doi:10.1080/00207543.2021.1926568.

[17] Zhu, Q., Yuan, J., Wang, G., Hou, Y., 2025. Scheduling a single-arm two-cluster tool with a process module failure subject to wafer residency time constraints. IEEE Transactions on Automation Science and Engineering 22, 667–679. doi:10.1109/TASE.2023.3275703.

[18] Kim, D., Kim, H.J., 2025. Noncyclic scheduling of cluster tools using deep reinforcement learning. IEEE Transactions on Automation Science and Engineering 22, 24009–24023. doi:10.1109/TASE.2025.3632534.

[19] Lee, J.H., Zhao, C., Li, J., Papadopoulos, C.T., 2018. Analysis, design, and control of Bernoulli production lines with waiting time constraints. Journal of Manufacturing Systems 46, 208–220. doi:10.1016/j.jmsy.2018.01.001.

[20] Kim, H.J., Lee, J.H., 2025. Scheduling cluster tools for concurrent processing: Deep reinforcement learning with adaptive search. IEEE Transactions on Automation Science and Engineering 22, 3783–3796. doi:10.1109/TASE.2024.3399818.

[21] Tang, Y., Agrawal, S., Faenza, Y., 2020. Reinforcement learning for integer programming: Learning to cut, in: Proceedings of the 37th International Conference on Machine Learning, PMLR. pp. 9367–9376. URL: https://proceedings.mlr.press/v119/tang20a.html.

[22] Huang, Z., Wang, K., Liu, F., Zhen, H.L., Zhang, W., Yuan, M., Hao, J., Yu, Y., Wang, J., 2022. Learning to select cuts for efficient mixed-integer programming. Pattern Recognition 123, 108353. doi:10.1016/j.patcog.2021.108353.

[23] Paulus, M.B., Zarpellon, G., Krause, A., Charlin, L., Maddison, C., 2022. Learning to cut by looking ahead: Cutting plane selection via imitation learning, in: Proceedings of the 39th International Conference on Machine Learning, PMLR. pp. 17584–17600. URL: https://proceedings.mlr.press/v162/paulus22a.html.

[24] Wang, Z., Li, X., Wang, J., Kuang, Y., Yuan, M., Zeng, J., Zhang, Y.,

Wu, F., 2023. Learning cut selection for mixed-integer linear programming via hierarchical sequence model, in: Proceedings of the Eleventh International Conference on Learning Representations. URL: https://openreview.net/forum?id=Zob4P9bRNcK.

[25] Achterberg, T., 2009. SCIP: Solving constraint integer programs. Mathematical Programming Computation 1, 1–41. doi:10.1007/s12532-008-0001-1.

[26] Bestuzheva, K., Besançon, M., Chen, W.K., Chmiela, A., Donkiewicz, T., van Doornmalen, J., Eifler, L., Gaul, O., Gamrath, G., Gleixner, A., Gottwald, L., Graczyk, C., Halbig, K., Hoen, A., Hojny, C., van der Hulst, R., Koch, T., Lübbecke, M.E., Maher, S.J., Matter, F., Mühmer, E., Müller, B., Pfetsch, M.E., Schlösser, F., Serrano, F., Shinano, Y., Sofranac, B., Turner, M., Vigerske, S., Wegscheider, F., Wellner, P., Weninger, D., Witzig, J., 2023. Enabling research through the SCIP optimization suite 8.0. ACM Transactions on Mathematical Software 49, 22:1–22:21. doi:10.1145/3585516.

[27] Wang, J., Wang, Z., Li, X., Kuang, Y., Shi, Z., Zhu, F., Yuan, M., Zeng, J., Zhang, Y., Wu, F., 2024. Learning to Cut via Hierarchical Sequence/Set Model for Efficient Mixed-Integer Programming. arXiv:2404.12638. URL: https://arxiv.org/abs/2404.12638.

[28] Vinyals, O., Fortunato, M., Jaitly, N., 2015. Pointer networks, in: Advances in Neural Information Processing Systems, pp. 2692–2700.

[29] Choo, J., Kwon, Y.D., Kim, J., Jae, J., Hottung, A., Tierney, K., Gwon, Y., 2022. Simulation-guided beam search for neural combinatorial optimization, in: Advances in Neural Information Processing Systems. URL: https://papers.nips.cc/paper_files/paper/2022/hash/39b9b60f0d149eabd1fff2d7c7d5afc4-Abstract-Conference.html, doi:10.52202/068431-0637.

[30] Nemhauser, G.L., Wolsey, L.A., Fisher, M.L., 1978. An analysis of approximations for maximizing submodular set functions—i. Mathematical Programming 14, 265–294. doi:10.1007/BF01588971.

[31] Williams, R.J., 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning 8, 229–256. doi:10.1007/BF00992696.

[32] Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T.P., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous methods for deep reinforcement learning, in: Proceedings of the 33rd International Conference on Machine Learning, PMLR. pp. 1928–1937. URL: https://proceedings.mlr.press/v48/mniha16.html.